Two-party ECDSA Signing at Constant Communication Overhead

Yashvanth Kondi
yash@ykondi.net
Silence Laboratories

October 3, 2025

Abstract

In this work, we investigate whether the cost of two-party ECDSA signing can be brought within the realm of plain ECDSA signing. We answer the question in the affirmative for the case of communication complexity, by means of a new signing protocol. Our protocol consumes bandwidth linear in the security parameter, and hence the size of an ECDSA signature. Our scheme makes only blackbox use of generic tools—Oblivious Transfer during key generation, and any Pseudorandom Function when signing. While computation complexity is not asymptotically optimal, benchmarks of our protocol confirm that concrete costs are the lowest known for ECDSA signing. Our protocol is therefore the most concretely efficient in the literature on all fronts: bandwidth, computation, and rounds.

On a technical level, our protocol is enabled by a novel Pseudorandom Correlation Function (PCF) for the Vector Oblivious Linear Evaluation correlation over a large ring. The PCF relies on one-way functions alone, and may be of independent interest.

Our scheme supports standard extensions, such as pre-signing, and including backup servers for key shares in a $(2, n)$ configuration.

Contents

1 Introduction

Threshold cryptography is the practice of distributing key material amongst a group of nodes, so that a minimum quorum is required in order to meaningfully operate the system. Threshold signing for instance allows for the decentralization of signing authority by distributing the signing key.

The most visible use case of threshold signing that has driven its adoption in recent years has been in the domain of cryptocurrency custody. It is common practice today to decentralize the signing of transactions by distributing the signing key, so that signatures can only be produced with the engagement of more than one party. One of the most widely used configurations today corresponds to two-party signing, as it implements a natural two-factor authentication pattern—this will be the focus of our paper. Note that "two-party signing" quite easily translates to the $(2, n)$ setting, where backup key shares are stored on additional nodes.

While the academic literature has produced several threshold signing schemes with various tradeoffs in efficiency, cryptographic assumptions, tightness of reductions, etc., arguably the most significant need of the cryptocurrency setting is compatibility with externally set standards. As transactions are validated within a much broader system, any threshold-signed transactions must be compliant with the format that validators expect. For historical reasons, ECDSA has been the default signature scheme, and so distributed ECDSA signing has found success in securing cryptocurrency wallets.

There are of course several potential use cases for threshold signing beyond the present ones. As ECDSA is by far the most ubiquitous elliptic curve based signature, and legacy compatibility in internet protocols is typically non-negotiable, the utility of threshold ECDSA signing is clear even outside of the cryptocurrency domain. However, the cost of existing distributed ECDSA signing protocols can be a barrier to adoption in many scenarios. The non-linear structure of ECDSA has entailed distributed signing protocols that make use of cryptographic machinery that is substantially heavier and more complex than ECDSA itself. While it is possible to trade costs between computation and bandwidth for instance by altering the machinery (eg. OT vs Paillier encryption), every two-party signing protocol in the literature incurs at least an order of magnitude overhead in every dimension relative to plain signing. This overhead may be justifiable in critical scenarios such as digital asset custody, but it poses a barrier for general ubiquitous use.

The goal of this work is to investigate if the efficiency of two-party ECDSA can be brought within the realm of plain ECDSA signing—i.e. within the same asymptotic complexity class, and, ideally, comparable concrete efficiency as well. Communication complexity serves as a good starting point, as it is easier to analyze than computation, which requires reasoning about operations in different groups, ciphers with hardware acceleration support, etc. As signing curves are widely believed not to permit attacks better than generic ones, ECDSA is typically instantiated with 2λ-bit sized curves for a computational security parameter of λ. We therefore define "constant overhead communication" for a

distributed ECDSA signing protocol to mean that it achieves communication that is linear in the security parameter as well. It is unclear how one might achieve this target even with powerful cryptographic tools, as the machinery that underlies existing ECDSA signing protocols—Oblivious Linear Evaluation (OLE)—appears to be inherently superlinear. Instantiating OLE in these works typically comes with a tradeoff: using generic tools such as Oblivious Transfer tends to be computationally lighter but more bandwidth intensive, whereas Additively Homomorphic Encryption (AHE) such as Paillier or Class Groups enable more succinct communication. However, even Paillier and Class Group ciphertexts are superlinear in the security parameter, as they permit attacks that are better than generic ones. Achieving constant communication overhead for distributed ECDSA therefore appears to require a fundamentally different approach.

This work. We introduce a new two-party ECDSA signing protocol that we assert makes it a prime candidate for wide deployment:

- **Efficiency.** Our protocol has communication complexity linear in the security parameter—achieving the constant communcation overhead target for the first time. Concretely, our most optimized signing protocol requires transmitting only 170 bytes, comparable to exchanging a pair of ECDSA signatures in each direction. While the computation required by our scheme is asymptotically greater than ECDSA, concrete costs are low—benchmarks of our protocol on a laptop averaged 2.2ms to sign a message. Our protocol is simultaneously cheaper in concrete computation and bandwidth than every prior work, while matching the best known round complexity.

- **Simple, ECDSA-Native Tools.** Our protocol only makes blackbox use of generic cryptography: Oblivious Transfer (OT) and Pseudorandom Functions (PRFs). Of these, OT is only used during key generation, and can be instantiated with Diffie-Hellman style techniques with the ECDSA signing curve. Signing only requires PRF evaluations, in addition to a couple of standard proofs of knowledge discrete logarithm. Besides the conceptual advantage, this enables flexible instantiations that can take advantage of hardware acceleration as available in different contexts.

The efficiency gains of our protocol are enabled by two technical insights that may be of independent interest beyond ECDSA signing. The first is a novel Pseudorandom Correlation Function (PCF) [BCG+20] for the Vector Oblivious Linear Evaluation (VOLE) correlation over a large ring $\mathbb{Z}_M$. The PCF follows from a structural similarity between Gilboa's OLE [Gil99] and IKNP OT extension [IKNP03] that enables us to leverage recent progress in OT extension to construct more efficient Vector OLE. In particular, our PCF consists of several instances of Roy's PCF for VOLE in small fields [Roy22] that are combined using the Chinese Remainder Theorem (CRT) into a single VOLE over a large ring. A similar approach was recently also used in the context of garbling arithmetic circuits [CHHK25].

While efficient PCFs for VOLE over large rings are already known from Paillier [OSY21, RS21], they have been difficult to use in the elliptic curve context. First, note that Paillier entails superlinear communication anyway. Second, PCFs from those works generate correlations modulo a semiprime N (and ours an even smoother composite M) whereas the elliptic curve context requires correlations modulo a prime q, the group order. This mismatch can be reconciled easily in the semi-honest setting. Intuitively, the parties can first adjust the $\mathbb{Z}_M$ correlations to their chosen inputs mod q, and assuming $M \gg q$ the adjusted correlation will not overflow M and therefore hold over the integers directly. Subsequently, each party can simply take its components of the correlation mod q to obtain the desired output, i.e. a $\mathbb{Z}_q$ VOLE correlation. However, the malicious setting introduces scope for a selective failure attack: a corrupt party can adjust the correlation incorrectly so that it may overflow M, i.e. the adjustment (and any protocol that uses the correlation) fails conditional on the honest partys secret. This issue is difficult to mitigate; for instance in the context of deterministic signing, Kondi et al. [KOR23] and Boneh et al. [BHLS25] achieved verifiable adjustment by resorting to expensive tools such as integer commitments and range proofs.

Solving the above issue is where our second insight comes in: we use semi-honest adjustment unmodified and just allow for the selective failure attack, and prove that it does not meaningfully harm unforgeability. Our proof is a generalization of a principle from Lindell's Paillier-based ECDSA signing protocol [Lin17], wherein aborts due to selective failure are simulated by a simple guessing strategy. We extract the fundamental idea that underlies their proof strategy, and show it to be agnostic to Paillier by applying it in our context as well. At a high level, the idea is that the "leakage" due to selective failure in the threshold signing context essentially corresponds to the index of the first session in which an abort is witnessed—there are only polynomially many such sessions, and so this information can be simulated (in the unforgeability experiment) with a random guess. We devise a simple approach to make use of this principle: we define our signing functionality to incorporate the selective failure clause, and we show that the unforgeability of its signatures reduces to that of ECDSA itself (albeit with a small security loss, which can optionally be avoided with an additional assumption). This design paradigm is not ECDSA-specific, and may find use in the design of threshold signing protocols for "MPC-unfriendly" signature schemes more generally.

We provide an efficiency comparison with a few representative prior works in Table 1.

1.1 Background

There are several works in the literature on threshold ECDSA signing that offer tradeoffs in computation and bandwidth consumption. While there has not necessarily been a clear line of evolution of techniques in the literature, Doerner et al. [DKLs24] recently put forward a framework by which to interpret most prior works. They proposed the following rough recipe for a threshold ECDSA

Protocol	Communication		Computation		Rounds
	Asymp.	KB	Dominant cost	(ms)	
[Lin17]	$\omega(\lambda)$	0.9	$O(1)$ Paillier	12	4
[CCL+19]	$\omega(\lambda)$	0.57	$O(1)$ Class groups	170	4
[CGG+20]	$\omega(\lambda)$	15	$O(1)$ Paillier	100+	4
[DKLs24]	$O(\lambda^2)$	115	$O(\lambda)$ OT[a]	4	3
[ABC+24]	$\omega(\lambda)$	2	$O(1)$ Paillier	48	2
Protocol 8.2	$O(\lambda)$	0.28	$O(\lambda^2/\log\lambda)$ PRF	2.2	3
Protocol 9.2	$O(\lambda)$	0.17	$O(\lambda^2/\log\lambda)$ PRF	2.2	2
Plain Signing	$O(\lambda)$	0.06	$O(1)$ $\mathbb{G}$	0.05	2

Table 1: Efficiency-alone comparison of the signing protocols in this work with a few representative prior works. We do not include key generation, or qualitative parameters such as assumptions or concurrent security. For instance, Protocol 9.2 and prior work by Adjedj et al. [ABC+24] require a *doubly enhanced* unforgeability assumption for ECDSA, and make use of the Knowledge of Exponent assumption, whereas other entries in the table do not. Empirical results are taken from benchmarks in different papers and are therefore not directly comparable, but they do provide an estimate of concrete computation costs.
[a] OT can be instantiated with OT extension protocols, so signing only requires hash evaluations.

protocol:

1. **Rewrite ECDSA Signing.** Securely computing the ECDSA equation most naturally suits arithmetic (i.e. large field) MPC, however a naive representation of ECDSA as just additions and multiplications in $\mathbb{Z}_q$ would be much too large. The first step therefore is to "rewrite" ECDSA in a manner that yields a suitably small arithmetic circuit.

2. **Cryptographic Machinery.** Once a rewriting for ECDSA has been established, one needs some kind of cryptographic machinery to securely compute it in a distributed fashion. Performing this computation even in the semi-honest setting is non-trivial as it entails secure multiplication. This step therefore corresponds to establishing a cryptographic mechanism to compute the ECDSA equation in a "private but unauthenticated" manner, meaning that secret shares of a putative signature are computed if parties follow the protocol. One may think of this step as plugging in an (unauthenticated) OLE or MtA gadget such as those based on Oblivious Transfer [DKLs18, DKLs24], or additively homomorphic encryption like Paillier [Lin17, GG18, CGG+20] or class groups [CCL+19, CCL+20].

3. **Verification.** Given that an honest execution of the protocol produces shares of a signature, most works then employ a mechanism to verify that all parties followed the protocol (up to simulatable erros). This is done in prior works with a variety of methods: statistical MACs [ST19, DOK+20, DKLs24], zero-knowledge proofs [CGG+20, ABC+24], replicating the computation in committed form [LN18], or more elaborate interactive protocols [GG18].

Doerner et al. [DKLs24] showed that the "ECDSA tuple" method of rewriting ECDSA [ANO+22] is conducive to a simple signing protocol that only makes blackbox use of VOLE and ideal commitments between each pair of parties. This framework yields very efficient instantiations, such as one that uses an OT-based VOLE in their work. However, the ECDSA tuple rewriting is not necessarily the best fit for all cases. There are two other rewritings of ECDSA: the "multiplicative" rewriting due to MacKenzie and Reiter [MR01], and the "inverted nonce" rewriting of Langford [Lan95] and Gennaro et al. [GJKR96]. The multiplicative rewriting in particular has yielded very competitive two-party signing protocols, such as Lindell's [Lin17] and its follow up works [ABC+24, BHLS25].

Lindell's protocol [Lin17] is based on Paillier encryption, and has the lowest known bandwidth consumption of any two-party ECDSA signing at around 1KB, while requiring about 11ms to generate a signature on standard hardware. This is 3× the signing time of the fastest known ECDSA protocol (based on OT [DKLs18, DKLs24]), and >4× faster than any other protocol based on Paillier. Despite being among the earlier practical threshold ECDSA signing protocols, it remains very relevant even today, and continues to be widely used. Seemingly, the only conceptual downside is that the protocol and its analysis make use of Paillier encryption in a rather non-blackbox way.

In this work, we extract and generalize the fundamental ideas behind Lindell's protocol, and provide a substantially more efficient instantiation based on general assumptions. In particular, we show that the underlying cryptographic machinery used by the protocol can be abstractly expressed as a "reactive VOLE", and the verification mechanism can in fact be generic. Whereas reactive VOLE is instantiated using Paillier encryption in Lindell's work, we give an instantiation that uses only OT during setup/key generation, and any PRF when signing. Our reactive VOLE exploits a connection between Gilboa's OLE technique [Gil99] and IKNP-style OT extension protocols [IKNP03], and leverages the recent small field VOLE due to Roy [Roy22] by composing many such VOLEs using the Chinese Remainder Theorem. As Roy's construction is essentially a PCF, our composed construction is a PCF as well.

Related Work on Optimizing OLE with CRT. Gilboa's protocol [Gil99] constructs OLE from OT at a cost that is quadratic in the size of the modulus for the arithmetic, i.e. $O(\ell^2)$ bits to multiply ℓ-bit values. Chen et al. [CCD+20] observed that performing a semi-honest OLE on ℓ-bit integers can be decomposed into $\ell/\log \ell$ OLEs on $\log \ell$ sized primes via CRT. Given that each of these $\ell/\log \ell$ smaller OLEs costs only $(\log \ell)^2$ bits, Chen et al. were able to achieve

OLE over an ℓ-bit ring at a cost of $O(\ell \log \ell)$ bits, not counting the cost of the OTs themselves. Upgrading their construction to malicious security while retaining their efficiency gains is difficult, as a naive approach would incur a security parameter $\lambda > \log \ell$ overhead in each of the OLEs. The very recent work of Doerner et al. [DHIM25] showed how to use techniques from number theory retain some of the efficiency of the semi-honest CRT optimization in the malicious setting, moreover for primes in addition to smooth moduli. The scope of our work differs from both of these prior works, because we do not need OLE per se—our setting is closer to *vector* OLE, where each signing instance corresponds to an element of the vector. The most important metric to us is being able to extend the vector, which we show for a well-structured ring $\mathbb{Z}_M$ can be done at *no communication* by leveraging recent OT extension techniques [Roy22]. We then translate the $\mathbb{Z}_M$ correlation to $\mathbb{Z}_q$ by communicating $O(|q|)$ bits, bringing our overall complexity to be linear in the modulus size. In summary, our focus is on reactive VOLE rather than standard OLE, for which previous works require quasilinear (in modulus size) communication per instance, which our work improves to linear for prime moduli, and zero communication for primorial moduli. Our prime modulus VOLE does not achieve full malicious security, which we show to be sufficient in our context.

2 Organization

In Section 3, we cover relevant prior works in deeper detail, and incrementally build our techniques against this background. After this overview, we proceed with formal descriptions.

We first give our security model and necessary preliminaries in Section 4. Next, in the interest of self-containment, we recap the small VOLE construction of Roy [Roy22] in Section 5. We then show how to compose several instances of small VOLE to construct a large $\mathbb{Z}_M$ VOLE in Section 6, along with a mechanism to enforce a bound on the size of the receiving party's input.

We present two variants of our ECDSA signing protocol: one whose security reduces to the standard unforgeability of ECDSA, and another whose security depends on a version of ECDSA with adversarial nonce bias. The latter permits a more efficient two-round signing protocol, at the expense of assuming a property of ECDSA that is shown to hold in the Generic Group Model [ABC+24] but is nonetheless nonstandard. Our fundamental construction may be seen as a "partial" ECDSA signing protocol, which can be wrapped in different ways as the application requires. For this reason, we present a partial ECDSA signing functionality and protocol in Section 7, which we show how to wrap to obtain a 3-message protocol for a standard signing functionality in Section 8, or a 2-message protocol for a biased signing functionality in Section 9. We prove that these functionalities yield unforgeable signatures in Section 10, and discuss the efficiency of their instantiations in Section 11. Finally, we conclude by briefly discussing some extensions of our results in Section 12.

3 Our Techniques and Their Evolution from Prior Works

We explain our insights by first explicitly casting Lindell's protocol in the earlier framework (i.e. rewriting, cryptographic machinery, and verification), and then showing how each component can be generalized, and finally giving our new instantiations. We will begin our discussion assuming that both parties follow the protocol, and then return to malicious adversaries only in the verification component in Section 3.3.

3.1 Multiplicative Rewriting of ECDSA

First, let us recall the structure of an ECDSA signature over an elliptic curve $\mathcal{G} = (\mathbb{G}, G, q)$, generated by G and of prime order q. A public key is a group element $\mathsf{pk} \in \mathbb{G}$ and its corresponding secret key $\mathsf{sk} \in \mathbb{Z}_q$ is its discrete logarithm, i.e. it satisfies $\mathsf{sk} \cdot G = \mathsf{pk}$. A signature σ consists of a randomly sampled signing nonce $R = k \cdot G$, and a scalar $s \in \mathbb{Z}_q$ such that:

$$s = \frac{H(m) + r_x \cdot \mathsf{sk}}{k} \pmod{q}$$

where r_x is the x-coordinate of R, and $H(m)$ is the hashed message, typically SHA2 in many standards. MacKenzie and Reiter [MR01] observed that the computation of s can be split into two halves—$H(m)/k$ and $r_x\mathsf{sk}/k$—of which two parties can derive *multiplicative* shares non-interactively. This idea was further refined by Lindell [Lin17] and Adjedj et al. [ABC+24] to require just one multiplication of secret values. Consider the following rewriting of ECDSA:

$$R = k_0 k_1 \cdot G \quad \text{and} \quad s = \frac{s_0}{k_0} \quad \text{where } s_0 = (\alpha \cdot \mathsf{sk}_0 + \beta)$$

$$\text{with } \alpha = \frac{r_x \cdot \mathsf{sk}_1}{k_1} \text{ and } \beta = \frac{H(m)}{k_1}$$

Observe that a party P_0 that knows sk_0, k_0 and is authorized to learn s, can safely be given s_0 assuming that the individual values α, β are withheld, as s_0 is fully determined by s and k_0. This yields a simple structure for a canonical two-party signing protocol, given an OLE oracle (whose functionality is implicit in the protocol below).

Canonical Two-party ECDSA with Multiplicative Rewriting:

1. P_0, P_1 sample sk_0, k_0 and sk_1, k_1 respectively
2. P_0, P_1 exchange pk_0, R_0 and pk_1, R_1 as Diffie-Hellman key exchange messages to determine $\mathsf{pk} = \mathsf{sk}_0\mathsf{sk}_1 \cdot G$ and $R = k_0 k_1 \cdot G$
3. They run an OLE where P_0 inputs sk_0 and P_1 inputs α, β to deliver $s_0 = \alpha \cdot \mathsf{sk}_0 + \beta$ to P_0

4. P_0 outputs the signature $(R, s_0/k_0)$

The OLE therefore corresponds to the cryptographic machinery to securely compute this rewriting of ECDSA. Lindell [Lin17] made use of a classic protocol of Gilboa's [Gil99] based on Additively Homomorphic Encryption for this purpose. Simply put, P_0 who owns a public key apk sends $\mathsf{ct}_0 = \mathsf{Enc}_{\mathsf{apk}}(\mathsf{sk}_0)$ to P_1, who then responds with $\mathsf{ct}_1 = \alpha \cdot \mathsf{ct} + \mathsf{Enc}_{\mathsf{apk}}(\beta)$. Due to the additive homomorphism of the encryption scheme, ct_1 is an encryption of $s_0 = \alpha\mathsf{sk}_0 + \beta$, allowing P_0 to decrypt ct_1 and obtain s_0 as required. Lindell additionally makes the useful observation that since P_0's input to the OLE is fixed, the same ct_0 value can be reused across multiple instances, and even preprocessed so it need only be sent once during key generation.

3.2 Generalizing the OLE Mechanism

While OLE based on AHE as above is conceptually simple, finding an appropriate AHE scheme to instantiate it in this context can be tricky. This is because constructing an AHE scheme that natively supports arithmetic in $\mathbb{Z}_q$ is challenging without exotic tools such as class groups; using common tools such as Paillier (as Lindell does) entails emulating $\mathbb{Z}_q$ arithmetic within a much larger composite modulus N.

Our first step is therefore to consider a more general functionality that achieves the effect of this OLE. One could of course use an OLE functionality, but we wish to capture all of the efficiency gains of Lindell's protocol; recall that the first message of the OLE (i.e. ct_0) is reused, which is not a standard OLE feature. At first glance, it may seem like *vector* OLE fits this need exactly, but there is a caveat: the number of instances across which ct_0 is reused (the size of the vector in VOLE) corresponds to the number of signatures, and so does not have an apriori bound. We therefore define a new *reactive* VOLE functionality that allows for this feature explicitly.

Reactive VOLE:

- **Receiver input:** Receive and store $(\texttt{input}, x)$ sent by P_0, notify P_1 and do not accept future instructions from P_0.
- **Sender input:** Upon receiving $(\texttt{send}, a, b)$ from P_1, if $(\texttt{input}, x)$ exists in memory, deliver $(\texttt{output}, y = ax + b)$ to P_0. This step may be executed an unlimited number of times.

The relationship between reactive VOLE and standard VOLE is not immediate. Of course, one may separate the two with contrived protocols, but it is an interesting question as to whether natural VOLE protocols can be safely used in reactive mode.

With the reactive VOLE abstraction in place, the next step is to consider alternative instantiations. In the same paper from which the AHE-based OLE is derived [Gil99], Gilboa also provides an alternative OT-based OLE

instantiation. We recall it briefly below:

Gilboa's OT-OLE:

- **Receiver input**(x): P_0 derives a bit decomposition $\{\mathbf{x}_i\}_{i\in[|q|]}$ of its input $x \in \mathbb{Z}_q$, and initiates $|q|$ instances of OT as the receiver, with $\mathbf{x}_i$ as its choice bit in the i^{th} instance.
- **Sender input**(a, b): P_1 samples $\{\mathbf{b}_i \leftarrow \mathbb{Z}_q\}_{i\in[|q|]}$ subject to $\sum 2^i \mathbf{b}_i = b$, and sends the message pair $(\mathbf{b}_i, a + \mathbf{b}_i)$ in the i^{th} OT instance. Party P_0 then receives $\mathbf{y}_i = \mathbf{b}_i + a\mathbf{x}_i$ from the i^{th} OT instance, and computes its output $y = \sum_i 2^i \mathbf{y}_i = (\sum_i 2^i \mathbf{b}_i) + (a \sum_i 2^i \mathbf{x}_i) = b + ax$.

Gilboa's OT-OLE lends itself easily to a reactive VOLE format, at least in the semi-honest setting. This follows from a well-known trick to extend the OT sender's message domain on the fly: the sender uses the OT itself to transmit a pair of symmetric encryption keys $\mathsf{k}_0, \mathsf{k}_1$ (of which the receiver obtains k_c per its choice bit c) so that when the sender wishes to transmit a fresh message pair μ_0, μ_1 for the same choice bit, sending $\mathsf{Enc}(\mathsf{k}_0, \mu_0), \mathsf{Enc}(\mathsf{k}_1, \mu_1)$ enables the receiver to decrypt μ_c. The cost of each fresh $\mathbb{Z}_q$ VOLE correlation derived through Gilboa's protocol is therefore $O(|q|^2)$ bits.

Gilboa's OT-OLE protocol already forms the basis for several ECDSA signing protocols, including those of Doerner et al. [DKLs18, DKLs19, DKLs24] as well as Haitner et al. [HMRT22, HLNR23]. Improving upon Gilboa's OT-OLE itself for arbitrary $\mathbb{Z}_q$ has been a long standing open problem. However, as Lindell observed, the efficiency of ECDSA *signing* in the multiplicative context corresponds to executing only the Sender Input phase, and so we need only concern ourselves with this step.

We show that the Sender Input phase of Gilboa's OT-OLE protocol can be significantly improved when it is used as a reactive VOLE. Readers familiar with the literature on OT extension may notice that when structured as a reactive VOLE, Gilboa's protocol is reminiscent of the celebrated IKNP technique [IKNP03]: in each Sender Input instance, P_1 transmits $|q|$ pairs of messages whose differences share the same correlation, and in the OT extension case $a \in \{0, 1\}$ which implies two high entropy values $\{\mathbf{y}_i\}_{i\in[|q|]}$ and $\{\mathbf{y}_i - \mathbf{x}_i\}_{i\in[|q|]}$. Both sets are known to P_0, and the a^{th} set is equal to $\{\mathbf{b}_i\}_{i\in[|q|]}$ as known to P_1. While Gilboa's OT-OLE makes use of the correlation quite differently from IKNP OT extension, we observe that recent improvements to IKNP in deriving the correlation itself carry over to Gilboa's technique in the reactive VOLE context.

The SoftSpokenOT protocol of Roy [Roy22] derived a computation-communication tradeoff in generating the correlation that underlies IKNP OT extension. The basis for the SoftSpokenOT technique is a VOLE construction for small $\mathbb{Z}_p$ (i.e. $p \in O(\text{poly}\,\lambda)$), which we recall below:

SoftSpokenVOLE(p): All arithmetic is done in $\mathbb{Z}_p$.

- **Receiver input:** P_0 and P_1 engage in a $\binom{p}{p-1}$-OT, where P_1 sends p fresh PRF keys $\{\mathsf{k}_i\}_{i\in\mathbb{Z}_p}$, and P_0 receives all but the x^{th} of them.
- **Sender input:** For a fresh identifier id, define $f_i = \mathsf{PRF}_p(\mathsf{k}_i, \mathsf{id})$. Then, party P_1 derives

$$w = \sum_{i\in\mathbb{Z}_p} i \cdot f_i \quad \text{and} \quad u = \sum_{i\in\mathbb{Z}_p} f_i$$

  while P_0 derives $v = \sum_{i\in\mathbb{Z}_p}(x-i)\cdot f_i$, which is possible even without f_x as its coefficient is zero anyway. Now observe that:

$$v + w = \sum_{i\in\mathbb{Z}_p} ((x-i)\cdot f_i + i\cdot f_i) = \sum_{i\in\mathbb{Z}_p} x\cdot f_i = x\cdot u$$

  As P_0 holds x, v and P_1 holds u, w such that $v + w = x\cdot u$, the VOLE correlation is achieved. Party P_1 can derandomize u, w to a, b by sending $\delta_a = a - u$ and $\delta_b = b - w$, and P_0 can offset v by adding $x\cdot\delta_a + \delta_b$.

Observe that underlying the SoftSpokenVOLE protocol above is a Pseudorandom Correlation Function (PCF) for the $\mathbb{Z}_p$ VOLE correlation; both parties derive their respective shares of the correlation non-interactively, and derandomize as required. However, due to difficulties inherent in working with PCF/PCG definitions [BCG+19], and the fact that we will need to derandomize the correlations before use in ECDSA signing, in our formalisms we choose to encapsulate the effect of SoftSpokenVOLE within a reactive *small* VOLE functionality $\mathcal{F}_{\mathsf{rsVOLE}}(p)$. While the VOLE protocol in [Roy22] is described for small fields, Kondi et al. [KOR23] observed that the same technique works for large $\mathbb{Z}_q$ with the caveat that the receiver's input can only be polynomially large. In particular, the SoftSpokenVOLE protocol can non-interactively generate correlations of the form $y = a\cdot x + b$ in $\mathbb{Z}_q$ for a fixed receiver input $x < p$, where $p \in O(\text{poly}(\lambda))$. Realizing a "full-sized" VOLE in $\mathbb{Z}_q$ can therefore be achieved by simply composing $|q|/|p|$ instances of SoftSpokenVOLE à la Gilboa. This already yields a substantial factor $\log|q|$ improvement; setting even $p = 256$ entails less than a millisecond of computation per $\mathbb{Z}_q$ VOLE, in exchange for shrinking bandwidth by nearly an order of magnitude.

This direct replacement of OTs in Gilboa's protocol with SoftSpokenVOLE still incurs asymptotic bandwidth redundancy—each VOLE instance transmits $O(|q|^2/\log|q|)$ bits due to having to derandomize the $|q|/\log|q|$ VOLE PCFs with a $\mathbb{Z}_q$ element each. Rather than directly following Gilboa's template, we show that one can instead combine a number of small $\mathbb{Z}_p$ VOLE PCFs into a single $\mathbb{Z}_M$ VOLE PCF using the Chinese Remainder Theorem. Then, derandomizing the $\mathbb{Z}_M$ VOLE requires transmitting only a pair of $\mathbb{Z}_M$ elements, which when $M \in O(|q|)$ as we show to be sufficient, achieves our constant overhead target.

Our approach borrows an idea from the way Paillier-based VOLE PCF has previously been adapted to work in the elliptic curve context [KOR23]. Intuitively, we make use of VOLE correlations within a large ring $\mathbb{Z}_M$, where M is

chosen such that simply reducing modulo q yields the correct correlation. The modulus M must be large enough to accommodate the integer value $ax+b+\eta\cdot q$ without overflowing, where $\eta \in \mathbb{Z}_{q^2}$ is an integer one-time pad that statistically masks any information beyond $ax + b \pmod q$. However, we operate with a modulus M that is conveniently structured, so that the $\mathbb{Z}_M$ VOLE correlations can be generated with just PRFs instead of public-key primitives like Paillier. In more detail, if $\mathbf{p} \in \mathbb{Z}^r$ is a vector of the first r primes, then $M = \prod_{i\in[r]} \mathbf{p}_i$ is the smallest primorial number such that $M > q^3+2q^2$, so that multiplication in $\mathbb{Z}_M$ is congruent to multiplication in $\mathbb{Z}_{\mathbf{p}_1} \times \cdots \times \mathbb{Z}_{\mathbf{p}_r}$. This is convenient, as each $\mathbf{p}_i$ is small enough to permit instantiation of $\mathcal{F}_{\mathsf{rsVOLE}}(\mathbf{p}_i)$ with SoftSpokenVOLE. With severe abuse of notation, one might say that:

$$\mathcal{F}_{\mathsf{rVOLE}}(M) \cong \mathcal{F}_{\mathsf{rsVOLE}}(\mathbf{p}_1) \times \cdots \times \mathcal{F}_{\mathsf{rsVOLE}}(\mathbf{p}_r)$$

This enables a dramatic efficiency improvement, as each $\mathcal{F}_{\mathsf{rsVOLE}}(\mathbf{p}_i)$ invocation transmits only two $\mathbb{Z}_{\mathbf{p}_i}$ elements, implying that each $\mathbb{Z}_M$ VOLE instance requires transmitting only a pair of $\mathbb{Z}_M$ elements. As M itself is roughly $3|q|$ bits long, the cost of each full-sized VOLE is now only $O(|q|)$ bits. We explicitly show how to achieve $\mathbb{Z}_q$ VOLE via $\mathbb{Z}_M$ below:

Reactive VOLE(q): Let r be the smallest integer such that $M > q^3 + 2q^2$, where $\mathbf{p} \in \mathbb{Z}^r$ are the first r primes, and $M = \prod_{i\in[r]} \mathbf{p}_i$.

- **Receiver input**(x): For each $i \in [r]$, party P_0 initiates an instance of $\mathcal{F}_{\mathsf{rsVOLE}}(\mathbf{p}_i)$ as the receiver, with $x \pmod{\mathbf{p}_i}$ as its input.
- **Sender input**(a, b): P_1 samples $\eta \leftarrow \mathbb{Z}_{q^2}$ and sets $b' = b+\eta\cdot q$. Then, for each $i \in [r]$, party P_1 sends the message pair $(a \pmod{\mathbf{p}_i}, b' \pmod{\mathbf{p}_i})$ to $\mathcal{F}_{\mathsf{rsVOLE}}(\mathbf{p}_i)$, inducing P_0 to receive

  $$\mathbf{y}_i = a \cdot x + b \pmod{\mathbf{p}_i}$$

  as its output from the i^{th} VOLE instance.

  Subsequently, P_0 computes $y' = \mathsf{CRTRecon}(\mathbf{y}, \mathbf{p}) \pmod q$, and finally outputs the value $y = y' \pmod q$.

By the Chinese Remainder Theorem, we have that $y' = ax + b' \pmod M$. Observe that since $ax + b < 2q^2$ and $\eta q < q^3$, it follows that $ax + b' < q^3 + 2q^2$. Given that $M > q^3 + 2q^2$, it holds that taking $ax + b' \pmod M$ does not overflow, i.e. the modulo operation preserves the integer value. Therefore, party P_0 outputs:

$$\begin{aligned} y &= (a \cdot x + b' \pmod M) \pmod q \\ &= a \cdot x + b' \pmod q \\ &= a \cdot x + (b + \eta \cdot q) \pmod q \\ &= a \cdot x + b \pmod q \end{aligned}$$

which is exactly the desired VOLE correlation.

As for security, we can show that any potential leakage provided to P_0 given $y' = ax + b'$ is harmless. We can characterize y' by its quotient and residue relative to q, i.e. $y' = y_{\mathsf{q}} \cdot q + y$ where $y = y' \pmod q$ and $y_{\mathsf{q}} = \lfloor y/q \rfloor$. As the residue y is the intended output, it only remains to analyze y_{q}. Given that that $y_{\mathsf{q}} = \lfloor (ax + b)/q \rfloor + \eta$ where η is uniform in $\mathbb{Z}_{q^2}$ and $\lfloor (ax + b)/q \rfloor \in \mathbb{Z}_q$, the value y_{q} is effectively a statistical one-time pad encryption of $\lfloor (ax + b)/q \rfloor$. In other words, y_{q} can be simulated by simply drawing a value from $\mathbb{Z}_{q^2}$ uniformly at random, implying a statistical security loss of roughly $q/q^2 = 1/q$, which is negligible.

As each $\mathbb{Z}_{\mathbf{p}_i}$ VOLE correlation is derived non-interactively prior to derandomization, the parties effectively derive a random correlation in $\mathbb{Z}_M$ non-interactively (at the start of the Sender input phase). This directly implies a PCF for the $\mathbb{Z}_M$ VOLE correlation, which may be of independent interest.

3.3 General Verification for Maliciously Secure 2P-Signing

Applying the three-stage recipe (rewriting, OLE, verification) that Doerner et al. [DKLs24] devised to interpret ECDSA signing protocols retrospectively, there is typically a clear component of each protocol that enforces honest behaviour. This includes zero-knowledge proofs, interactive verification that parties hold shares of a valid signature, information-theoretic MACs, etc., each with their own costs and merits. However, Lindell's signing protocol is essentially just a passively secure protocol augmented by signature verification at the end (basic proofs of knowledge of discrete logarithms notwithstanding).

While the idea is superficially intuitive—an MPC that computes a signature can be checked by verifying the signature itself—actually proving security requires some care, evidenced by failed approaches, and the use of heavier machinery in other works. At a high level, assuming that the parameters for the Paillier-based OLE are verified during key generation, the only scope for malicious behaviour during signing is for P_1 to send a Paillier ciphertext that was not honestly computed. Efficiently enforcing the well-formedness of Paillier ciphertexts in the OLE context is notoriously challenging; some protocols were found to be insecure in their design or implementation [TS21, MYG24], and others employ zero-knowledge proofs that either entail orders of magnitude computational overhead [CGG+20], or aggressive optimization with small exponent assumptions [ABC+24]. With this background, it is remarkable that Lindell's protocol gets away without any explicit verification of Paillier ciphertexts during signing.

We extract the essence of why Lindell's protocol is resilient to malformed Paillier ciphertexts, and generalize the idea to show that it is applicable to our protocol as well—simply using the semi-honest Reactive VOLE from Section 3.2 and checking if the output is a valid ECDSA signature suffices for malicious security when signing. At a high level, a malformed ciphertext in Lindell's protocol leads to one of two outcomes:

- **Case 1:** P_0 outputs a valid ECDSA signature.
- **Case 2:** P_0 does not obtain a valid ECDSA signature, and therefore aborts and refuses to engage in further signing sessions.

At first glance, it appears that the adversary learns one bit of useful information by observing whether the outcome was successful or not, commonly known as a selective failure attack. Indeed, if the abort clause as per Case 2 is not properly implemented, the adversary can extract the full secret key after sufficiently many ignored failures [MYG24]. However, assuming that aborts are faithful to the protocol description, the adversary does not gain meaningful advantage through this channel. An intuitive comparison can be made to maliciously secure OT extension protocols [KOS15, ALSZ17, Roy22] wherein an adversarial receiver may cheat and learn ℓ bits of the honest sender's secret correlated encryption key, but with a $2^{-\ell}$ probability of aborting the protocol (and therefore preventing further leakage). Observe that essentially the same advantage can be achieved by simply guessing the bits; the "leakage" via the protocol does not meaningfully help an attacker. A rough analogy may be drawn to the ECDSA signing case as well: the probability with which an attacker induces an abort when attempting to learn a bit of the secret key, exactly corresponds to the uncertainty of that bit.

While the above argument may be intuitive, it is challenging to translate it to a proof strategy in the ECDSA context. Unlike the OT extension setting where the leakage is effectively on random oracle queries (which is useless unless the entire query is reconstructed), in the ECDSA setting the leakage is on elliptic curve scalars, which admit more structure and therefore could potentially be more dangerous. The principle that underlies the proof in [Lin17] is actually quite simple: consider an adversary that participates in up to T signing instances. At most one of these instances will abort due to malformed Paillier ciphertexts. Therefore, the only information that can be gleaned through the signing protocol executions is exactly *which* of the T instances will be the one to abort. This does not in fact provide meaningful advantage in forgery—as $T \in O(\text{poly}(\lambda))$, the index of the aborting session is only $O(\log \lambda)$ bits of information. A forger for the two-party ECDSA protocol therefore yields a forger for ECDSA itself, with $O(\log \lambda)$ bits of security loss (the index of the aborting session can simply be guessed at random). Concretely, even across say a hundred thousand two-party signatures, the security loss in this proof strategy is about 16 bits, which for a 256-bit curve means a security level comparable to a 2048-bit RSA key. We stress that this loss is due to the proof strategy, and not an actual attack; later on in [Lin17] a custom assumption permits a tight reduction.

The above logic is embedded in the monolithic simulation strategy of Lindell's protocol. However, it is clearly agnostic to the use of tools like Paillier. We present a more general version of this idea, which may be of independent interest. Consider the following weakening of the unforgeability experiment, which we call *predicated unforgeability*:

- **Setup:** Sample key pair $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KeyGen}(1^\lambda)$, and send pk to $\mathcal{A}$.
- **Signing oracle:** On receiving a message m and predicate ϕ from $\mathcal{A}$, if $\phi(\mathsf{sk}) = 0$ then return $\mathsf{Sign}(\mathsf{sk}, m)$. Otherwise do not respond, and disable this oracle for the rest of the experiment.
- **Output:** Output 1 if the adversary produces a valid signature on a message that it did not query to the signing oracle.

Let (Q, ε) denote the number of signing queries, and success probability of an adversary respectively. The exact same logic now applies:

Theorem 3.1. *(Informal). A (Q, ε) adversary $\mathcal{A}$ for predicated unforgeability implies a $(Q, \varepsilon/Q)$ adversary $\mathcal{A}'$ for standard unforgeability.*

As sketched earlier, the reduction is achieved by simply guessing a random signing oracle query at which to abort. Given that predicated unforgeability is practically as good as standard unforgeability, we relax our target functionality to *predicated* ECDSA signing:

Predicated 2P-ECDSA Signing:

- **Setup:** Upon request from P_0 and P_1, sample a key pair $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{ECDSAGen}(1^\lambda)$ and send pk to both parties.
- **Signing m:** Upon request from P_0, sample nonce $k \leftarrow \mathbb{Z}_q$ and send $R = k \cdot G$ to P_1. Wait for predicate ϕ from P_1, and if $\phi(\mathsf{sk}) = 0$ then send $\mathsf{ECDSASign}(\mathsf{sk}, m; k)$ to P_0. Otherwise, abort and ignore future queries.

The diligent reader might notice that the above functionality allows P_1 to choose its predicate after seeing the signing nonce. This has no impact on the proof strategy or security bounds, as guessing the index of the signing query at which to abort succeeds all the same—we formally show this in Section 10. We also formulate an assumption of a similar flavour to Lindell's ad-hoc assumption, that yields a tight reduction for our protocol in Section 12.3. Simply stated, the assumption is that conditioning the abort on evaluating ϕ on a dummy secret key sk' rather than the actual secret key sk, is undetectable to an efficienct adversary. The statement of the assumption depends on the predicate; while Lindell's assumption is phrased as an interaction between elliptic curves and Paillier, ours is expressed with the elliptic curve alone.

Additionally, we show that the functionality even permits a small degree of bounded concurrency; even if the abort clause is not propagated across $O(1)$ concurrent instances in time (perhaps due to an operational oversight), the unforgeability guarantee remains with tolerable security loss.

3.4 Putting it Together

With predicated ECDSA in place as our target functionality, we now explain how our $\mathcal{F}_{\mathsf{rsVOLE}}$ based signing protocol achieves this notion. The canonical protocol

for two-party ECDSA signing with the multiplicative rewriting (Section 3.1) is already maliciously secure when using a secure $\mathbb{Z}_q$ VOLE. The difference with our protocol is that it uses a $\mathbb{Z}_M$ VOLE to emulate the effect of one in $\mathbb{Z}_q$. We argue below that if the $\mathbb{Z}_M$ VOLE is used incorrectly, then it implies a well-defined ϕ that the simulator can extract and provide to the predicated ECDSA signing functionality.

First, observe that if all inputs to the $\mathbb{Z}_M$ VOLE are in the correct range, then the $\mathbb{Z}_q$ VOLE is emulated exactly, and we are done. However, we need to reason about the scenario in which either a malicious P_0 or P_1 use inputs that are too large.

Malicious P_0. The only scope for P_0 to cheat is to provide an incorrect sk_0 value during the setup phase. Ensuring that P_0's input sk_0 is correct essentially entails proving that it is in the correct range, and that $\mathsf{sk}_0 \cdot G = \mathsf{pk}$. This is a little involved, but executed only once in the entire protocol (during key generation), and so we defer the exact protocol description to Section 6.

Malicious P_1. Recall that we only target a *predicated* signing functionality—a fact that we can leverage to simulate the adversary's view. First, observe that as per the canonical signing protocol in Section 3.1, P_1's inputs to the $\mathbb{Z}_q$ VOLE are fully and uniquely specified: they ought to be $\alpha = r_x \mathsf{sk}_1 / k_1$ and $\beta = H(m)/k_1$. Next, observe that once pk, R, m are fixed, there is a unique scalar s that will complete the signature, and correspondingly a unique $\mathbb{Z}_q$ value $s_0 = \alpha \cdot \mathsf{sk}_0 + \beta$ that P_0 ought to obtain in an honest execution of the protocol. In other words, if we denote P_1's input to the $\mathbb{Z}_M$ VOLE as (a, b), then the protocol will succeed if and only if it holds that:

$$y = (a \cdot \mathsf{sk}_0 + b \pmod M) \pmod q = \alpha \cdot \mathsf{sk}_0 + \beta \pmod q$$

which clearly implies a predicate ϕ on the signing key sk for the simulator to submit to the predicated signing interface.

Therefore, we have constructed a two-party signing protocol that realizes the predicated ECDSA functionality—which provably inherits the unforgeability of ECDSA itself—at the cost of just one VOLE in a conveniently structured ring $\mathbb{Z}_M$ (and two standard proofs of knowledge of discrete logarithm). This effectively entails transmitting a pair of $\mathbb{Z}_M$ elements aggregated across all $\mathcal{F}_{\mathsf{rsVOLE}}$ invocations—about $6|q|$ bits in total—and a small constant number of curve points and scalars for the proofs of knowledge of discrete logarithm. Computation is dominated by $\sum_{i \in [r]} \mathbf{p}_i \in O(\lambda^2 / \log \lambda)$ invocations of PRF, which can be instantiated with AES or any cipher with an accelerated implementation within the deployment context. In Section 11 we discuss optimizations and concrete costs in more detail.

4 Preliminaries

Security Model. We use the simulation based security paradigm of Canetti [Can00], which guarantees sequential composability. All of our simulators are straightline, and therefore allow for concurrent composition as well. In fact, our unbiased signing protocol is even Universally Composable (UC) [Can01] when the underlying building blocks (particularly the zero-knowledge proofs) are UC-secure. Our biased signing protocol is not UC secure as written due to a technicality: it makes use of the Knowledge of Exponent assumption [BP04, "KEA1"], in which extraction of secrets depends on the adversary's code. Note that the ideal functionalities that our protocols realize are of *predicated* signing as discussed earlier. While previous work permits predicated signing implicitly within a game-based definition [Lin17], our ideal functionality makes it explicit for clarity.

Our signing protocols require all executions to be halted in the event that an abort is detected—even concurrent ones, although we prove some tolerance to overriding this clause in Theorem 10.2. This paradigm, followed by Lindell's protocol [Lin17] and called "global abort" in the same work, is also used in OT extension based protocols [KOS15, DKLs18, Roy22, DKLs24], and MPC protocols more broadly [DPSZ12, KOS16]. Care must be taken to ensure that aborts are properly handled, and there are now several implementation-oriented resources to this end [LL23, vdP23]. However unlike OT extension based protocols where aborts are conditioned on OT extension state (which is independent of the signing key) [DKLs18, DKLs19, DKLs24], in our protocol and Lindell's (original global abort variant), aborts are conditional on the signing key itself. This implies that it is prudent to re-key in the event that cheating is detected.

Helper Functionalities. Our protocols make use of $\binom{p}{p-1}$ Oblivious Transfer, where p is a flexible parameter. This is encapsulated in a functionality $\mathcal{F}_{\mathsf{OT}}$. We also make use of functionalities for zero-knowledge $\mathcal{F}_{\mathsf{ZK}}$ and committed zero-knowledge $\mathcal{F}_{\mathsf{ComZK}}$, which allows a prover to first commit to the statement and witness, and reveal the statement at a later point. In both cases, the relation corresponds to the standard proof of knowledge of discrete logarithm. As these are standard functionalities, we relegate them to Appendix A.

Assumptions. Our biased signing protocol makes use of the Knowledge of Exponent assumption [BP04, "KEA1"] assumption, which is that for any PPT adversary $\mathcal{A}$, there exists an extractor $\mathcal{E}_{\mathcal{A}}$ such that the following event occurs with negligible probability:

$$(X, Y) = (x{\cdot}G, x{\cdot}Y) \wedge x \neq x' : (G, H) \leftarrow \mathbb{G}^2, (X, Y) \leftarrow \mathcal{A}(G, H),\ x' \leftarrow \mathcal{E}_{\mathcal{A}}(G, H)$$

Additionally, our biased signing protocol assumes Doubly Enhanced Unforgeability of ECDSA [ABC+24], which we recall in Appendix B.

ECDSA. For an elliptic curve $\mathcal{G} = (\mathbb{G}, G, q)$ that consists of a group $\mathbb{G}$ generated by G of prime order q, an ECDSA key pair consists of a secret $\mathsf{sk} \leftarrow \mathbb{Z}_q$ and public $\mathsf{pk} = \mathsf{sk} \cdot G$. Signing a message begins by sampling $k \leftarrow \mathbb{Z}_q$ and setting r_x as the x-coordinate of $R = k \cdot G$, and completing the signature with the equation $\mathsf{ECDSASign}(\mathsf{sk}, m; k) = (H(m) + \mathsf{sk} \cdot r_x)/k$. The signature itself consists of the tuple (R, s).

5 Reactive Small VOLE

We begin by recalling the small VOLE construction due to Roy [Roy22]. First, we describe the ideal functionality below.

Functionality 5.1. $\mathcal{F}_{\mathsf{rsVOLE}}(p)$**: Reactive Small VOLE**

This functionality interacts with two parties, S and R, whom we refer to as the sender and the receiver respectively. It also interacts directly with the ideal adversary $\mathcal{S}$, who can instruct the functionality to abort at any time. It is parameterized by a prime p, which defines the modulus for the arithmetic.
The sender can provide arbitrarily many inputs, each of which are securely applied to the receiver's committed input.

Receiver Input: On receiving $(\texttt{r-input}, \mathsf{sid}, x)$ from R such that sid is previously unused for any input, and $x \in \mathbb{Z}_p$, send $(\texttt{r-input}, \mathsf{sid})$ to S, and store $(\texttt{r-input}, \mathsf{sid}, x)$ in memory once S responds with $(\texttt{s-ack}, \mathsf{sid})$.

Sender Input: On receiving $(\texttt{s-input}, \mathsf{sid}, \mathsf{ssid}, a, b)$ from S such that $(\texttt{r-input}, \mathsf{sid}, x)$ exists in memory, ssid is previously unused for any input, and $a, b \in \mathbb{Z}_p$, send $(\texttt{r-output}, \mathsf{sid}, \mathsf{ssid}, y = a \cdot x + b \pmod p)$ to R.

Now, we are ready to give the protocol itself.

Protocol 5.2. $\pi_{\mathsf{rsVOLE}}(p, q)$**: Reactive Small VOLE**

This protocol is executed between two parties, S and R, whom we refer to as the sender and the receiver respectively. Assuming an implicit security parameter λ, the protocol is parameterized by a prime $p \in O(\mathrm{poly}(\lambda))$.

The sender can provide arbitrarily many inputs, each of which are securely applied to the receiver's input, which is fixed in all executions.
The protocol makes use of an ideal $\binom{p}{p-1}$-OT oracle $\mathcal{F}_{\mathsf{OT}}$, and a pseudorandom function $\mathsf{PRF} : \{0,1\}^\lambda \times \{0,1\}^{2\lambda} \mapsto \mathbb{Z}_q$.

Receiver Input: On R being provided with private input $(\texttt{r-input}, \mathsf{sid}, x)$ such that sid is previously unused for any input, and $x \in \mathbb{Z}_p$,

1. R sends $(\mathsf{sid}, \texttt{choose-all-but}, x)$ to $\mathcal{F}_{\mathsf{OT}}$

2. On receiving $(\mathsf{sid}, \texttt{chosen})$ from $\mathcal{F}_{\mathsf{OT}}$, party S samples and stores $\{\mathsf{k}_i \leftarrow \{0,1\}^\lambda\}_{i\in\mathbb{Z}_p}$, and sends $(\mathsf{sid}, \texttt{messages}, \{\mathsf{k}_i\})$ to $\mathcal{F}_{\mathsf{OT}}$.

3. R receives $(\mathsf{sid}, \texttt{outputs}, \{\mathsf{k}_i\}_{i\in\mathbb{Z}_p\setminus\{x\}})$ from $\mathcal{F}_{\mathsf{OT}}$

Sender Input: On S being provided with private input $(\texttt{s-input}, \mathsf{sid}, \mathsf{ssid}, a, b)$ with $a, b \in \mathbb{Z}_q$ and a fresh ssid,

1. S sets $u_p = 0$, and computes $\{u_i = \mathsf{PRF}(\mathsf{k}_i, \mathsf{ssid}) + u_{i+1}\}_{i\in\mathbb{Z}_p}$ and $u = \sum_{i\in\mathbb{Z}_p} u_i$

2. S computes $\delta_a = a - u_0$ and $\delta_b = b - (u_0 - u)$ and sends $(\mathsf{sid}, \mathsf{ssid}), (\delta_a, \delta_b)$ to R.
Note: session identifiers (sid,ssid*) can be omitted if clear from context, such as in the protocols in this paper.*

3. R sets $v_p = 0$, and computes $\{v_i = \mathsf{PRF}(\mathsf{k}_i, \mathsf{ssid}) + v_{i+1}\}_{i\in\mathbb{Z}_p}$, with the exception that $v_x = v_{x+1}$, and sets:

$$v = \sum_{i\in\mathbb{Z}_p} v_i \quad \text{and} \quad y' = v_0 \cdot x - v$$

4. R outputs $(\texttt{r-output}, \mathsf{sid}, \mathsf{ssid}, y = y' + x \cdot \delta_a + \delta_b)$

Theorem 5.3. *Assuming* PRF *is a pseudorandom function, Protocol* $\pi_{\mathsf{rsVOLE}}(p)$ *realizes functionality* $\mathcal{F}_{\mathsf{rsVOLE}}(p)$ *in the* $\mathcal{F}_{\mathsf{OT}}$*-hybrid model in the presence of an active adversary statically corrupting up to one party.*

We refer the reader to Roy [Roy22] for the proof.

6 Large VOLE with Range Check

In this section, we provide our $\mathbb{Z}_M$ VOLE functionality, along with its realization. Besides implementing the standard VOLE interface over $\mathbb{Z}_M$, our functionality additionally ensures that the following properties on the receiver's input x at setup time:

1. It is within the correct range, i.e. $x < 2^{|q|}$

2. It corresponds to the discrete logarithm of a public value, i.e. $x \cdot G = X$

The first property is achieved by a simple range proof. We execute a passive version of Gilboa's OT-OLE protocol, wherein R inputs each bit of x into an OT, which R and S use to derive OLE correlations for each $\mathbf{p}_i$ separately. The OLE correlations are compared against those derived via $\mathcal{F}_{\mathsf{rsVOLE}}$, to verify that they match. In particular, R obtains $a \cdot x + b \pmod{\mathbf{p}_i}$ via Gilboa's protocol, as well as $a \cdot x + \hat{b} \pmod{\mathbf{p}_i}$ via $\mathcal{F}_{\mathsf{rsVOLE}}$, and takes their difference to obtain

$\delta = b - \hat{b} \pmod{\mathbf{p}_i}$—a value that S can derive independently. As this check has soundness error $1/\mathbf{p}_i$, it is repeated $\lambda/|\mathbf{p}_i|$ times for full security. As Gilboa's protocol enforces a $2^{|q|}$ bound on R's input size (by virtue of guaranteeing a well-defined bit decomposition), and the mechanism described above enforces equality of inputs between the Gilboa OLE and $\mathcal{F}_{\mathsf{rsVOLE}}$ instances, the range bound is achieved.

The second property is achieved by a standard technique wherein a random $\mathbb{Z}_q$ OLE correlation $ax + b$ is checked "in the exponent", so that it matches $a \cdot X + b \cdot G$.

As our VOLE will be used in a context that permits an abort conditional on R's input, we allow for this provision in our VOLE functionality as well. This enables security against a malicious S almost for free. In order to account for a corrupt S during the range check, we have S provide $z = \mathsf{RO}(\mathbf{\Delta})$ to R, so R may non-interactively verify that it has derived the correct $\mathbf{\Delta}$ values before sending them to S. This commits S to an efficiently extractable predicate on R's input, permitting simulation relative to the predicated functionality.

We give the functionality first, and then the protocol below.

Functionality 6.1. $\mathcal{F}_{\mathsf{rVOLE}}(\mathcal{G}, \mathsf{s})$**: Reactive Oversized VOLE**

This functionality interacts with two parties, S and R, whom we refer to as the sender and the receiver respectively. It also interacts directly with the ideal adversary $\mathcal{S}$, who can instruct the functionality to abort at any time. It is parameterized by an integer s and elliptic curve $\mathcal{G}$ generated by $G \in \mathbb{G}$ and of prime order q. All arithmetic in this functionality is performed in $\mathbb{Z}_M$, where M is the smallest primorial number larger than $2^{\mathsf{s}} \cdot q^2$.
The sender can provide arbitrarily many inputs, each of which are securely applied to the receiver's committed input.

Receiver Input: On receiving $(\texttt{r-input}, \mathsf{sid}, x)$ from R such that sid is previously unused for any input, and $x < 2^{|q|}$, send $(\texttt{r-committed}, \mathsf{sid}, x \cdot G)$ to S. In case S is corrupt, wait for $(\texttt{predicate}, \mathsf{sid}, \phi)$ from S, and send $(\texttt{abort}, \mathsf{sid})$ to R if $\phi(x) = 0$. Store $(\texttt{r-input}, \mathsf{sid}, x)$ in memory if there is no abort, and send $(\texttt{success}, \mathsf{sid})$ to S.

Sender Input: On receiving $(\texttt{s-input}, \mathsf{sid}, \mathsf{ssid}, a, b)$ from S such that $(\texttt{r-input}, \mathsf{sid}, x)$ exists in memory, ssid is previously unused for any input, and $a, b \in \mathbb{Z}_M$, send $(\texttt{r-output}, \mathsf{sid}, \mathsf{ssid}, y = a \cdot x + b \pmod{M})$ to R.

We now give the protocol.

Protocol 6.2. $\pi_{\mathsf{rVOLE}}(\mathcal{G}, \mathsf{s})$**: Reactive** $\mathbb{Z}_M$ **VOLE**

This protocol is executed between two parties, S and R, whom we refer to as the sender and the receiver respectively. It is parameterized by an statistical security parameter s and elliptic curve $\mathcal{G}$ generated by $G \in \mathbb{G}$ and of prime order q. Let $\mathbf{p} \in \mathbb{Z}^r$ be the first r primes so that $M = \prod_{i \in [r]} \mathbf{p}_i$ is the smallest primorial number larger than $2^{\mathsf{s}} \cdot q^2$. Additionally, we define

$\{\mathbf{r}_i = \lceil \lambda / \log_2 \mathbf{p}_i \rceil\}_{i \in [r]}$ where λ is the computational security parameter. The sender can provide arbitrarily many inputs, each of which are securely applied to the receiver's input, which is fixed in all executions.
The protocol makes use of an ideal $\binom{2}{1}$-OT oracle $\mathcal{F}_{\mathsf{OT}}$, small VOLE oracles $\mathcal{F}_{\mathsf{rsVOLE}}(\mathbf{p}_i)$ for each $i \in [r]$, and a collection of random oracles indexed by their range, i.e. $\mathsf{RO}_p : \{0,1\}^* \mapsto \mathbb{Z}_p$.

Receiver Input: On R being provided with private input $(\texttt{r-input}, \mathsf{sid}, x)$ such that sid is previously unused for any input, and $x \in \mathbb{Z}_q$,

1. R defines $\{\mathbf{x}_j \in \{0,1\}\}_{j \in \{0..|q|-1\}}$ to be the bit decomposition of x.
2. R transmits the following values:
   (a) To S: the curve point $X = x \cdot G$
   (b) To $\mathcal{F}_{\mathsf{OT}}$: $(\mathsf{sid}^j_{\mathsf{OT}}, \texttt{choose}, \mathbf{x}_j)$ for each $j \in \{0..|q|-1\}$
   (c) To $\mathcal{F}_{\mathsf{rsVOLE}}(\mathbf{p}_i)$ for each $i \in [r]$: $(\texttt{r-input}, \mathsf{sid}^i_{\mathsf{sVOLE}}, x \pmod{\mathbf{p}_i})$

   which S waits to receive either directly, or be notified of through the appropriate ideal functionality.
3. S generates its check values:
   (a) Sample $\alpha \leftarrow \mathbb{Z}_q$ and $\beta \leftarrow \mathbb{Z}_{2^s \cdot q^2}$, and set $\Gamma = \alpha \cdot X + \beta \cdot G$
   (b) Sample $\boldsymbol{\mu} = \{\boldsymbol{\mu}_{j,0}, \boldsymbol{\mu}_{j,1} \leftarrow \mathbb{Z}_q^2\}_{j \in [0..|q|-1]}$
   (c) For each $i \in [r]$,
      i. Sample $\{\mathbf{a}_{i,\ell}, \hat{\mathbf{b}}_{i,\ell} \leftarrow \mathbb{Z}^2_{\mathbf{p}_i}\}_{\ell \in [\mathbf{r}_i]}$
      ii. For each $j \in \{0..|q|-1\}$, $\ell \in [\mathbf{r}_i]$, define the following values:

$$\mathbf{b}_{i,j,\ell} = \mathsf{RO}_{\mathbf{p}_i}(\texttt{mul}, i, \ell, \boldsymbol{\mu}_{j,0})$$
$$\mathbf{ct}_{i,j,\ell} = \mathsf{RO}_{\mathbf{p}_i}(\texttt{mul}, i, \ell, \boldsymbol{\mu}_{j,1}) - (\mathbf{a}_{i,\ell} + \mathbf{b}_{i,j,\ell}) \pmod{\mathbf{p}_i}$$

      iii. Set $\boldsymbol{\Delta}_i = \{\boldsymbol{\Delta}_{i,\ell} = \hat{\mathbf{b}}_{i,\ell} - \sum_{j \in [0..|q|-1]} 2^j \cdot \mathbf{b}_{i,j,\ell} \pmod{\mathbf{p}_i}\}_{\ell \in [\mathbf{r}_i]}$
      Note that $\boldsymbol{\Delta}_i \in \mathbb{Z}^{\mathbf{r}_i}_{\mathbf{p}_i}$ is at least λ bits long.
   (d) Set $\boldsymbol{\Delta} = \{\boldsymbol{\Delta}_i\}_{i \in [r]}$, and then $z = \mathsf{RO}_q(\texttt{commit}, \boldsymbol{\Delta}, \Gamma)$
4. S then transmits the following values:
   (a) To $\mathcal{F}_{\mathsf{OT}}$: $(\mathsf{sid}^j_{\mathsf{OT}}, \texttt{messages}, (\boldsymbol{\mu}_{j,0}, \boldsymbol{\mu}_{j,1}))$ for each $j \in \{0..|q|-1\}$
   (b) To $\mathcal{F}_{\mathsf{rsVOLE}}(\mathbf{p}_i)$ for each $i \in [r]$,
      i. $(\texttt{s-input}, \mathsf{sid}^i_{\mathsf{sVOLE}}, \mathsf{ssid}_X, \alpha \pmod{\mathbf{p}_i}, \beta \pmod{\mathbf{p}_i})$
      ii. For each $\ell \in [\mathbf{r}_i]$: $(\texttt{s-input}, \mathsf{sid}^i_{\mathsf{sVOLE}}, \mathsf{ssid}_\ell, \mathbf{a}_{i,\ell}, \hat{\mathbf{b}}_{i,\ell})$

(c) To R: $\mathbf{ct} = \{\mathbf{ct}_{i,j,\ell}\}_{i\in[r],j\in\{0..|q|-1\},\ell\in[\mathbf{r}_i]}$ and z

5. R waits to receive:
   (a) From $\mathcal{F}_{\mathsf{OT}}$: $(\mathsf{sid}^j_{\mathsf{OT}}, \mathtt{message}, \boldsymbol{\mu}_{j,\mathbf{x}_j})$ for each $j \in \{0..|q|-1\}$
   (b) From $\mathcal{F}_{\mathsf{rsVOLE}}(\mathbf{p}_i)$, for each $i \in [r]$:
      i. $(\mathtt{r\text{-}output}, \mathsf{sid}^i_{\mathsf{sVOLE}}, \mathsf{ssid}_X, \boldsymbol{\gamma}_i)$
      ii. For each $\ell \in [\mathbf{r}_i]$: $(\mathtt{r\text{-}output}, \mathsf{sid}^i_{\mathsf{sVOLE}}, \mathsf{ssid}_\ell, \hat{\mathbf{y}}_{i,\ell})$
   (c) From S: $\mathbf{ct} = \{\mathbf{ct}_{i,j,\ell}\}_{i\in[r],j\in\{0..|q|-1\},\ell\in[\mathbf{r}_i]}$ and z
6. R does the following for each $i \in [r]$:
   (a) For each $\ell \in [\mathbf{r}_i]$, set

$$\mathbf{y}_{i,\ell} = \sum_{j\in\{0..|q|-1\}} 2^j \cdot (\mathsf{RO}_{\mathbf{p}_i}(\mathtt{mul}, i, \ell, \boldsymbol{\mu}_{j,\mathbf{x}_j}) - \mathbf{x}_j \cdot \mathbf{ct}_{i,j,\ell}) \pmod{\mathbf{p}_i}$$

   (b) Set $\boldsymbol{\Delta}^{\mathsf{R}}_i = \{\boldsymbol{\Delta}^{\mathsf{R}}_{i,\ell} = \hat{\mathbf{y}}_{i,\ell} - \mathbf{y}_{i,\ell} \pmod{\mathbf{p}_i}\}_{\ell\in[\mathbf{r}_i]}$

   following which it assembles $\boldsymbol{\Delta}^{\mathsf{R}} = \{\boldsymbol{\Delta}^{\mathsf{R}}_i\}_{i\in[r]}$
7. R assembles $\Gamma^{\mathsf{R}} = \mathsf{CRTRecon}(\boldsymbol{\gamma}, \mathbf{p}) \cdot G$
8. R verifies that $z = \mathsf{RO}_q(\mathtt{commit}, \boldsymbol{\Delta}^{\mathsf{R}}, \Gamma^{\mathsf{R}})$, and sends $\boldsymbol{\Delta}^{\mathsf{R}}, \Gamma^{\mathsf{R}}$ to S
9. S upon receiving $\boldsymbol{\Delta}^{\mathsf{R}}, \Gamma^{\mathsf{R}}$ verifies that it matches $\boldsymbol{\Delta}, \Gamma$, and completes the setup phase successfully.

Sender Input: On S being provided with private input $(\mathtt{s\text{-}input}, \mathsf{sid}, \mathsf{ssid}, a, b)$ with $a, b \in \mathbb{Z}_M$ and a fresh ssid,

1. For each $i \in [r]$, S sends $(\mathtt{s\text{-}input}, \mathsf{sid}, \mathsf{ssid}_i, (a \mod \mathbf{p}_i), (b \mod \mathbf{p}_i))$ to $\mathcal{F}_{\mathsf{rsVOLE}}(\mathbf{p}_i)$
2. For each $i \in [r]$, R obtains $(\mathtt{r\text{-}output}, \mathsf{sid}, \mathsf{ssid}_i, \mathbf{y}_i \in \mathbb{Z}_{\mathbf{p}_i})$ from $\mathcal{F}_{\mathsf{rsVOLE}}(\mathbf{p}_i)$
3. R outputs $y = \mathsf{CRTRecon}(\{\mathbf{y}_i, \mathbf{p}_i\}_{i\in[r]})$

Theorem 6.3. *Protocol $\pi_{\mathsf{rVOLE}}(\mathcal{G}, \mathsf{s})$ realizes functionality $\mathcal{F}_{\mathsf{rVOLE}}(\mathcal{G}, \mathsf{s})$ in the $\mathcal{F}_{\mathsf{OT}}, \mathcal{F}_{\mathsf{rsVOLE}}, \mathsf{RO}$-hybrid model, in the presence of a malicious adversary statically corrupting up to one party.*

Proof. We divide our proof into two cases: corrupt R, and corrupt S.

Corrupt R. We describe the simulator $\mathcal{S}_{\mathsf{R}^*}$ below.

1. **Setup**. On receiving $(\mathtt{r\text{-}input}, \mathsf{sid}^i_{\mathsf{sVOLE}}, x \pmod{\mathbf{p}_i})$ on behalf of $\mathcal{F}_{\mathsf{rsVOLE}}$,

reconstruct $x \in \mathbb{Z}_M$ by $\mathsf{CRTRecon}$. Execute the rest of the setup phase by running the code of honest S. If $x < 2^{|q|}$, and $x \cdot G = X$, and no abort has occurred, then send $(\texttt{r-input}, \mathsf{sid}, x)$ to $\mathcal{F}_{\mathsf{rVOLE}}$. Otherwise, terminate with an abort.

2. **Sender Input**. On receiving $(\texttt{r-output}, \mathsf{sid}, \mathsf{ssid}, y)$ from $\mathcal{F}_{\mathsf{rVOLE}}$, for each $i \in [r]$ send $(\texttt{r-output}, \mathsf{sid}, \mathsf{ssid}_i, y \pmod{\mathbf{p}_i})$ to R on behalf of $\mathcal{F}_{\mathsf{rsVOLE}}(\mathbf{p}_i)$.

Lemma 6.4. *The distribution of transcripts produced by $\mathcal{S}_{\mathsf{R}^*}$ is computationally indistinguishable from that of the real protocol.*

The simulation differs from the real protocol in the setup phase only in the event that the inputs that R provides to $\mathcal{F}_{\mathsf{rsVOLE}}$ imply a reconstructed $\mathbb{Z}_M$ value $x \geq 2^{|q|}$ or $x \cdot G \neq X$, but S does not abort (whereas $\mathcal{S}_{\mathsf{R}^*}$ always does). We argue through the following hybrid experiments, that this event occurs with negligible probability.

Hybrid Experiment $\mathcal{H}_1^{\mathsf{R}}$. Define $x^* = \sum_{j \in [0..|q|-1]} 2^j \mathbf{x}_j$, where $\mathbf{x}_j$ is the input that R sends to the j^{th} $\mathcal{F}_{\mathsf{OT}}$ instance. Also define $\mathbf{y}$ such that for each $i \in [r]$ and $\ell \in [\mathbf{r}_i]$,

$$\mathbf{y}_{i,\ell} = \sum_{j \in \{0..|q|-1\}} 2^j \cdot (\mathsf{RO}_{\mathbf{p}_i}(\texttt{mul}, i, \ell, \boldsymbol{\mu}_{j,\mathbf{x}_j}) - \mathbf{x}_j \cdot \mathbf{ct}_{i,j,\ell}) \pmod{\mathbf{p}_i}$$

Then in this experiment, $\mathbf{ct}$ is sampled uniformly subject to

$$\mathbf{y}_{i,\ell} = \mathbf{a}_{i,\ell} \cdot x^* + \sum_{j \in \{0..|q|-1\}} 2^j \cdot \mathbf{b}_{i,j,\ell} \pmod{\mathbf{p}_i}$$

for each $i \in [r]$ and $\ell \in [\mathbf{r}_i]$. Note that the above relationship already holds in the real protocol. This change is detectable only if any $(\texttt{mul}, i, \ell, \boldsymbol{\mu}_{j,1-\mathbf{x}_j})$ is queried to $\mathsf{RO}_{\mathbf{p}_i}$, which occurs with negligible probability as $\boldsymbol{\mu}_{j,1-\mathbf{x}_j}$ is not present in R's view.

Hybrid Experiment $\mathcal{H}_2^{\mathsf{R}}$. In this experiment, the value $\boldsymbol{\Delta}$ is defined to be $\{\boldsymbol{\Delta}_i\}_{i \in [r]}$ such that

$$\boldsymbol{\Delta}_i = \{\boldsymbol{\Delta}_{i,\ell} = \hat{\mathbf{y}}_{i,\ell} - \mathbf{y}_{i,\ell} + (x^* - x) \cdot \mathbf{a}_{i,\ell} \pmod{\mathbf{p}_i}\}_{\ell \in [\mathbf{r}_i]}$$

This change is merely syntactic; recall that by virtue of $\mathcal{F}_{\mathsf{rsVOLE}}(\mathbf{p}_i)$, it holds that $\hat{\mathbf{y}}_{i,\ell} = \mathbf{a}_{i,\ell} \cdot x + \hat{\mathbf{b}}_{i,\ell} \pmod{\mathbf{p}_i}$, and $\mathbf{y}_{i,\ell}$ is as defined in the previous hybrid experiment.

Hybrid Experiment $\mathcal{H}_3^{\mathsf{R}}$. This experiment is the same as the last, except that it aborts with certainty when $x \neq x^*$. We argue below that this change is distinguishable with probability at most $T/2^{\lambda}$, where T is the running time of the distinguisher.

Consider any query of the form $\mathsf{RO}_q(\texttt{commit}, \mathbf{\Delta}^{\mathsf{R}}, \Gamma^{\mathsf{R}})$ made to RO_q in its final step of the setup phase. This query is the correct one, and experiment $\mathcal{H}_2^{\mathsf{R}}$ does not abort only if

$$\mathbf{\Delta}^{\mathsf{R}}_{i,\ell} = \mathbf{\Delta}_{i,\ell} = \hat{\mathbf{y}}_{i,\ell} - \mathbf{y}_{i,\ell} + (x^* - x) \cdot \mathbf{a}_{i,\ell} \pmod{\mathbf{p}_i}$$

in all indices. Observe that $\mathbf{y}, \hat{\mathbf{y}}, x, x^*$ are known to R, whereas $\mathbf{a}$ is not present in R's view at all. Also, if $x \neq x^*$, then there must exist at least one index $i' \in [x]$ such that $(x^* - x) \neq 0 \pmod{\mathbf{p}_{i'}}$. This implies that there is exactly one choice of $\mathbf{a}_{i',\ell} \in \mathbb{Z}_{\mathbf{p}_{i'}}^{\mathbf{r}_{i'}}$ for which the above equation will hold. The probability that this exact choice of $\mathbf{a}_{i',\ell}$ occurs is $(\mathbf{p}_i)^{-\mathbf{r}_i} \leq 2^{-\lambda}$, and so taking a union bound across T queries the chance that any of them are successful is at most $T/2^{\lambda}$.

Note that a direct implication of the above fact is that $x < 2^{|q|}$, as $x^* < 2^{|q|}$ by definition.

Hybrid Experiment $\mathcal{H}_4^{\mathsf{R}}$. Define $\gamma = \mathsf{CRTRecon}(\{\alpha \cdot x + \beta \pmod{\mathbf{p}_i}\}_{i \in [r]}, \mathbf{p})$, and observe that in Step 5(b)i of π_{rVOLE}, functionality $\mathcal{F}_{\mathsf{rsVOLE}}(\mathbf{p}_i)$ essentially delivers $\gamma \pmod{\mathbf{p}_i}$ to R. This experiment is the same as the last except that it samples a value $\gamma^* \leftarrow \mathbb{Z}_{2^{\mathsf{s}}q^2}$ subject to $\gamma = \gamma^* \pmod{q}$, and instead of Step 5(b)i of π_{rVOLE} it sends $(\texttt{r-output}, \mathsf{sid}^i_{\mathsf{sVOLE}}, \mathsf{ssid}_X, \gamma^* \pmod{\mathbf{p}_i})$. This change is distinguishable with probability at most $2^{-\mathsf{s}+1}$, as we argue below.

Given that $\alpha < q$, the value $\alpha \cdot x$ does not exceed q^2. Moreover, $\alpha \cdot x + \beta$ does not overflow M (except with probability $2^{-\mathsf{s}}$). Consequently, given that β is drawn uniformly from $\mathbb{Z}_{2^{\mathsf{s}}q^2}$, it functions as a statistical one-time pad; the values γ^* and γ are distributed identically in the range $[\alpha x, 2^{\mathsf{s}}q^2]$, and either value falls out of this range only with probability $(q^2/q^2 2^{\mathsf{s}}) = 2^{-\mathsf{s}}$. Note that the only evidence of α, β in R's view is encapsulated in $\Gamma = (\alpha \cdot x + \beta) \cdot G$, which is unchanged as $\gamma^* = \gamma \pmod{q}$.

Hybrid Experiment $\mathcal{H}_5^{\mathsf{R}}$. This experiment is the same as the last, except that it aborts with certainty when $x \cdot G \neq X$. The only distinguishing event between the two experiments is when $X = (x + \varepsilon) \cdot G$ for some nonzero ε, and R queries the correct Γ value to RO_q. This occurs with probability at most T/q where T is the running time of the distinguisher. The value $\alpha x + \beta \pmod{q}$ is distributed independently of α as $\beta \pmod{q}$ is uniform, implying that the probability that any Γ^{R} queried by the distinguisher matches

$$\Gamma = \alpha \cdot X + \beta \cdot G = \gamma^* \cdot G - \varepsilon\alpha \cdot G$$

is at most $1/q$. Taking a union bound over at most T attempts, the probability the correct query is made (which corresponds to the distinguishing probability between this experiment and the last) is at most T/q.

Given that protocol π_{rVOLE} is distributed indistinguishably from $\mathcal{H}_5^{\mathsf{R}}$—which implements the abort conditions of $\mathcal{S}_{\mathsf{R}^*}$—it follows that the setup phase of $\mathcal{S}_{\mathsf{R}^*}$ is distributed indistinguishably from the real protocol. The Sender Input phase of $\mathcal{S}_{\mathsf{R}^*}$ is simply a syntactic change from the real protocol by virtue of $\mathcal{F}_{\mathsf{rsVOLE}}$ and the Chinese Remainder Theorem. The lemma is hence proven.

Corrupt S. We describe the simulator $\mathcal{S}_{\mathsf{S}^*}$ below.

1. **Setup**.

   (a) On receiving $(\texttt{r-committed}, \mathsf{sid}, X)$ from $\mathcal{F}_{\mathsf{rVOLE}}$, send X to S and notify S on behalf of $\mathcal{F}_{\mathsf{OT}}$ and $\mathcal{F}_{\mathsf{rsVOLE}}$ that R's inputs have been committed.

   (b) On receiving the following values:

   - $\{\boldsymbol{\mu}_{j,0}, \boldsymbol{\mu}_{j,1}\}_{j\in[0..|q|-1]}$ on behalf of $\mathcal{F}_{\mathsf{OT}}$
   - The $\mathbb{Z}_M$ values (α, β) on behalf of $\mathcal{F}_{\mathsf{rsVOLE}}$ (reconstructed via CRT)
   - $\{\mathbf{a}_{i,\ell}, \hat{\mathbf{b}}_{i,\ell}\}_{i\in[r],\ell\in[\mathbf{r}_i]}$ on behalf of $\mathcal{F}_{\mathsf{rsVOLE}}(\mathbf{p}_i)$
   - $\mathbf{ct} = \{\mathbf{ct}_{i,j,\ell}\}_{i\in[r],j\in\{0..|q|-1\},\ell\in[\mathbf{r}_i]}$ and z on behalf of R

   search the list of random oracle queries made by S to locate $Q = (\texttt{commit}, \boldsymbol{\Delta}, \Gamma)$ such that $\mathsf{RO}_q(Q) = z$, aborting if not found.

   (c) Define the predicate $\phi(x)$ as follows:

   i. Let $\mathbf{x}$ be the bit decomposition of x

   ii. For each $\ell \in [\mathbf{r}_i]$, define

   $$\mathbf{y}_{i,\ell} = \sum_{j\in\{0..|q|-1\}} 2^j \cdot (\mathsf{RO}_{\mathbf{p}_i}(\texttt{mul}, i, \ell, \boldsymbol{\mu}_{j,\mathbf{x}_j}) - \mathbf{x}_j \cdot \mathbf{ct}_{i,j,\ell}) \pmod{\mathbf{p}_i}$$

   iii. Define $\boldsymbol{\Delta}_i^{\mathsf{R}} = \{\boldsymbol{\Delta}_{i,\ell}^{\mathsf{R}} = \hat{\mathbf{y}}_{i,\ell} - \mathbf{y}_{i,\ell} \pmod{\mathbf{p}_i}\}_{\ell\in[\mathbf{r}_i]}$

   iv. Define $\boldsymbol{\Delta}^{\mathsf{R}} = \{\boldsymbol{\Delta}_i^{\mathsf{R}}\}_{i\in[r]}$

   v. Define $\gamma = \alpha x + \beta \pmod{M}$, and then $\Gamma^{\mathsf{R}} = \gamma \cdot G$

   vi. Output 1 if $(\Delta, \Gamma) = (\Delta^{\mathsf{R}}, \Gamma^{\mathsf{R}})$ and 0 otherwise.

   and send $(\texttt{predicate}, \mathsf{sid}, \phi)$ to $\mathcal{F}_{\mathsf{rVOLE}}$.

   (d) If $(\texttt{success}, \mathsf{sid})$ is received from $\mathcal{F}_{\mathsf{rVOLE}}$, then send $\boldsymbol{\Delta}, \Gamma$ to S.

2. **Sender Input**. On receiving $(\texttt{s-input}, \mathsf{sid}, \mathsf{ssid}_i, \mathbf{a}_i, \mathbf{b}_i)$ on behalf of $\mathcal{F}_{\mathsf{rsVOLE}}(\mathbf{p}_i)$ for each $i \in [r]$, define $a = \mathsf{CRTRecon}(\mathbf{a}, \mathbf{p})$ and $b = \mathsf{CRTRecon}(\mathbf{b}, \mathbf{p})$, and send $(\texttt{s-input}, \mathsf{sid}, \mathsf{ssid}, a, b)$ to $\mathcal{F}_{\mathsf{rVOLE}}$.

Lemma 6.5. *The distribution of transcripts produced by $\mathcal{S}_{\mathsf{S}^*}$ is computationally indistinguishable from that of the real protocol.*

Observe that S's view in the real protocol consists of two components: control messages from $\mathcal{F}_{\mathsf{OT}}$,$\mathcal{F}_{\mathsf{rsVOLE}}$ notifying S that R's inputs have been committed, and Δ, Γ. The former are trivially identical in both executions, and so we focus on the latter. First, note that unless a collision is found in RO_q (happens with probability $T^2/q = T^2/2^{2\lambda}$ in running time T), there is a unique (Δ, Γ) that constitute a pre-image to z, and therefore a unique $(\Delta^{\mathsf{R}}, \Gamma^{\mathsf{R}})$ that R may send in the real protocol. Next, observe that R derives and sends these $(\Delta^{\mathsf{R}}, \Gamma^{\mathsf{R}})$ if and only if the predicate $\phi(x) = 1$—this predicate simply replicates the steps

of the honest R. Therefore, $\mathcal{S}_{\mathsf{S}^*}$ obtains and sends the exact same $(\Delta^{\mathsf{R}}, \Gamma^{\mathsf{R}})$ as R does, under exactly the same conditions (except in the negligibly likely event of a collision in RO_q). This implies that the simulation of the setup phase is computationally indistinguishable from the real protocol. The Sender Input phase is merely a syntactic change in the simulation, by virtue of $\mathcal{F}_{\mathsf{rsVOLE}}$ and the Chinese Remainder Theorem.

This proves the lemma, and hence the theorem.

□

6.1 Pseudorandom Correlation Function for $\mathbb{Z}_M$ VOLE

A Pseudorandom Correlation Function (PCF) [BCG+20] allows a pair of parties who are given correlated seeds, to locally expand these seeds to *arbitrarily many* pseudorandom values that satisfy a specified correlation. It is easy to see that such a PCF for the $\mathbb{Z}_M$ VOLE correlation is implicit in Protocol 6.2 above. We do not formally prove this however, as our protocols work in the simulation based security paradigm, where there are inherent difficulties in modeling such non-interactive objects [BCG+19]. Moreover, PCFs are not usually end goals in themselves; they are typically used in interactive protocols, which require the seeds to be sampled securely, and a clean, composable interface—both of which we provide.

Our construction above is therefore a contribution of independent interest, as to our knowledge it is the first PCF for VOLE in a large ring from one-way functions alone.

7 Partial ECDSA Signing from Reactive $\mathbb{Z}_M$ VOLE

With our $\mathbb{Z}_M$ VOLE in place, we are ready to construct the mechanism that underlies our ECDSA signing protocols, i.e. *partial* ECDSA signing. Essentially, partial signing takes as input the signing nonce share k_1 from P_1, and then delivers the signature share s_1 to P_0, so all that remains to be done to obtain a full signature is to divide by k_0. Full ECDSA signing then only requires a mechanism to sample k_0, k_1—this allows for flexible instantiations such as the plain version in Section 8, biased signing in Section 9, or even something more exotic such as an Exponent VRF [BHLS25].

As discussed earlier, we realize a *predicated* version of ECDSA signing, and this is reflected in our partial signing functionality as well. In particular, a corrupt P_1 may submit a predicate ϕ along with each query, which will induce an abort in the event that $\phi(\mathsf{sk}_0) = 0$.

We first give the partial ECDSA signing functionality, and then the protocol.

Functionality 7.1. $\mathcal{F}^{\mathsf{CA}}_{\mathsf{pECDSA}}(\mathcal{G})$: Partial ECDSA Signing

This functionality interacts with two parties, P_0, P_1. It also interacts di-

rectly with the ideal adversary $\mathcal{S}$, who can instruct the functionality to abort at any time. It is parameterized by an elliptic curve $\mathcal{G} = (\mathbb{G}, G, q)$ of prime order q and generated by G. As this is a two-party functionality, all outputs are adversarially delayed.

Setup: On receiving $(\texttt{setup}, \mathsf{sid})$ from P_0 such that sid is previously unused for any input:

1. If P_0 is corrupt, receive $(\texttt{share}, \mathsf{sid}, \mathsf{sk}_0 \in \mathbb{Z}_q)$ from $\mathcal{S}$, otherwise sample $\mathsf{sk}_0 \leftarrow \mathbb{Z}_q$.
2. Set $\mathsf{pk}_0 = \mathsf{sk}_0 \cdot G$, and send $(\texttt{ready}, \mathsf{sid})$ to P_1.
3. If P_1 is corrupt:
   - Wait for $(\texttt{share}, \mathsf{sid}, \mathsf{sk}_1 \in \mathbb{Z}_q)$ from $\mathcal{S}$, and send $(\texttt{pk-share}, \mathsf{sid}, \mathsf{pk}_0)$ in response.
   - Wait for $(\texttt{predicate}, \mathsf{sid}, \phi)$ from $\mathcal{S}$, and if $\phi(\mathsf{sk}) = 0$ then send $(\texttt{abort}, \mathsf{sid})$ to P_0 and halt.

   Otherwise, sample $\mathsf{sk}_1 \leftarrow \mathbb{Z}_q$.
4. Set $\mathsf{sk} = \mathsf{sk}_0 \cdot \mathsf{sk}_1$ and $\mathsf{pk} = \mathsf{sk} \cdot G$, and store $(\mathsf{sid}, \mathsf{sk})$.
5. Send $(\texttt{pk-out}, \mathsf{sid}, \mathsf{pk})$ to P_0 and P_1.

Partial Signing: On receiving a message of the form $(\texttt{sign-with}, \mathsf{sid}, \mathsf{sigid}, (k_1, h, r_x) \in \mathbb{Z}_q^3)$ from party P_1, if $(\mathsf{sid}, \mathsf{sk})$ exists in memory and sigid is previously unused:

1. If P_1 is corrupt, wait to receive $(\texttt{predicate}, \mathsf{sid}, \mathsf{sigid}, \phi_{\mathsf{sigid}})$ from $\mathcal{S}$. If $\phi_{\mathsf{sigid}}(\mathsf{sk}) = 0$, then send $(\texttt{abort}, \mathsf{sid}, \mathsf{sigid})$ to P_0 and halt.
2. Compute partial signature $s_1 = (h + (\mathsf{sk}_0 \cdot \mathsf{sk}_1) \cdot r_x) / k_1 \pmod q$ and send $(\texttt{partial-sig}, \mathsf{sid}, \mathsf{sigid}, h, r_x, k_1 \cdot G, s_1)$ to P_0.

Protocol 7.2. $\pi_{\mathsf{pECDSA}}(\mathcal{G}, \mathsf{s})$**: Partial Two-party ECDSA**

This protocol is executed between two parties, P_0, P_1. It is parameterized by an elliptic curve $\mathcal{G} = (\mathbb{G}, G, q)$ of prime order q and generated by G, and a statistical security parameter s. Let $\mathbf{p} \in \mathbb{Z}^r$ be the first r primes so that $M = \prod_{i \in [r]} \mathbf{p}_i$ is the smallest primorial number larger than $2^{\mathsf{s}} \cdot q^2$.

The protocol makes use of ideal oracles $\mathcal{F}_{\mathsf{ComZK}}$, $\mathcal{F}_{\mathsf{ZK}}$, and $\mathcal{F}_{\mathsf{rVOLE}}(\mathcal{G}, \mathsf{s})$ oracle. Let H denote the hash function used by ECDSA.

Setup: On P_0 receiving $(\texttt{setup}, \mathsf{sid})$ such that sid is previously unused for any input,

1. P_1 samples $\mathsf{sk}_1 \leftarrow \mathbb{Z}_q$, sets $\mathsf{pk}_1 = \mathsf{sk}_1 \cdot G$, and sends $(\texttt{prove}, \mathsf{sid}, \mathsf{sk}_1, \mathsf{pk}_1)$ to $\mathcal{F}_{\mathsf{ComZK}}$
2. Upon receiving $(\texttt{committed}, \mathsf{sid})$ from $\mathcal{F}_{\mathsf{ComZK}}$, P_0 samples $\mathsf{sk}_0 \leftarrow \mathbb{Z}_q$, sets $\mathsf{pk}_0 = \mathsf{sk}_0 \cdot G$, and sends $(\texttt{r-input}, \mathsf{sid}, \mathsf{sk}_0)$ to $\mathcal{F}_{\mathsf{rVOLE}}$
3. P_1 waits to receive $(\texttt{r-committed}, \mathsf{sid}, \mathsf{pk}_0)$, following which it sends $(\texttt{release}, \mathsf{sid})$ to $\mathcal{F}_{\mathsf{ComZK}}$
4. Upon receiving $(\texttt{released}, \mathsf{sid}, \mathsf{pk}_1)$ from $\mathcal{F}_{\mathsf{ComZK}}$, P_0 outputs the public key $\mathsf{pk} = \mathsf{sk}_0 \cdot \mathsf{pk}_1$.

Partial Signing: On input $(\texttt{sign-with}, \mathsf{sid}, \mathsf{sigid}, (k_1, h, r_x) \in \mathbb{Z}_q^3)$, party P_1 does the following:

1. Set $\alpha = (\mathsf{sk}_1 \cdot r_x)/k_1 \pmod q$ and $\beta = h/k_1 \pmod q$
2. Sample $\eta \leftarrow \mathbb{Z}_{q \cdot 2^s}$ and set $\beta' = \beta + q \cdot \eta$
3. Send $(\texttt{s-input}, \mathsf{sid}, \mathsf{sigid}, \alpha, \beta')$ to $\mathcal{F}_{\mathsf{rVOLE}}$
4. Send $(\texttt{prove}, \mathsf{sid}, \mathsf{sigid}||1, R_1, k_1)$ to $\mathcal{F}_{\mathsf{ZK}}$
5. Send (sigid, h, r_x) to P_0
   Note: session data (ssid, h, r_x) *need not be explicitly sent if clear from context, such as in protocols* π_{ECDSA} *and* π_{ECDSA}.

Finalize: P_0 does the following:

6. Wait to receive $(\texttt{r-output}, \mathsf{sid}, \mathsf{sigid}, s_1' \pmod M)$ from $\mathcal{F}_{\mathsf{rVOLE}}$, and $(\texttt{proven}, \mathsf{sid}, \mathsf{sigid}, R_1)$ from $\mathcal{F}_{\mathsf{ZK}}$
7. Compute the putative partial signature $s_1 = s_1' \pmod q$
8. Verify that $s_1 \cdot R_1 = h \cdot G + r_x \cdot \mathsf{pk}$, and output (R_1, s_1)

Theorem 7.3. *Protocol* $\pi_{\mathsf{pECDSA}}(\mathcal{G}, \mathsf{s})$ *realizes functionality* $\mathcal{F}^{\mathsf{CA}}_{\mathsf{pECDSA}}(\mathcal{G})$ *in the* $\mathcal{F}_{\mathsf{ComZK}}$,$\mathcal{F}_{\mathsf{ZK}}$,$\mathcal{F}_{\mathsf{rVOLE}}(\mathcal{G}, \mathsf{s})$*-hybrid model in the presence of a malicious adversary statically corrupting up to one party.*

Proof. We give two simulators $\mathcal{S}_{P0^*}$ and $\mathcal{S}_{P1^*}$, to simulate the view of the adversary based on the corruption of parties P_0 and P_1 respectively.

Corrupt P_0. The simulator $\mathcal{S}_{P0^*}$ proceeds as follows:

- **Setup.**
  1. Send $(\texttt{committed}, \mathsf{sid})$ to P_0, and wait to receive integer $\mathsf{sk}_0 < 2^{|q|}$ from P_0 on behalf of $\mathcal{F}_{\mathsf{rVOLE}}$.

2. Send $(\mathtt{share}, \mathsf{sid}, \mathsf{sk}_0 \pmod q)$ to $\mathcal{F}_{\mathsf{pECDSA}}^{\mathsf{CA}}$, and receive $(\mathtt{pk\text{-}out}, \mathsf{sid}, \mathsf{pk})$ in response.
3. Set $\mathsf{pk}_1 = \mathsf{sk}_0^{-1} \cdot \mathsf{pk}$, and send $(\mathtt{released}, \mathsf{sid}, \mathsf{pk}_1)$ to P_0 on behalf of $\mathcal{F}_{\mathsf{ComZK}}$.

- **Partial signing.** On receiving $(\mathtt{partial\text{-}sig}, \mathsf{sid}, \mathsf{sigid}, h, r_x, R_1, s_1)$ from $\mathcal{F}_{\mathsf{pECDSA}}^{\mathsf{CA}}$, sample $\eta' \leftarrow \mathbb{Z}_{q \cdot 2^s}$ and set $s_1' = s_1 + q \cdot \eta'$. Then, send the following values to P_0
  - $(\mathtt{proven}, \mathsf{sid}, \mathsf{sigid}, R_1)$ on behalf of $\mathcal{F}_{\mathsf{ZK}}$
  - (sigid, h, r_x) on behalf of P_1
  - $(\mathtt{r\text{-}output}, \mathsf{sid}, \mathsf{sigid}, s_1')$ to P_0 on behalf of $\mathcal{F}_{\mathsf{rVOLE}}$

Lemma 7.4. *The transcripts generated by $\mathcal{S}_{P0^*}$ are statistically indistinguishable from those of a real execution.*

The setup phase is distributed identically in the real and simulated protocols; the values $(\mathsf{pk}_0, \mathsf{pk}_1, \mathsf{pk})$ are essentially uniform Diffie-Hellman triples, and the control messages from ideal functionalities do not contain any information about their private inputs. In the partial signing phase, the only difference is in the way s_1' is computed: in $\mathcal{S}_{P0^*}$ this value is $s_1 + q \cdot \eta'$ whereas in the real protocol this value is of the form $s_1 + q \cdot (v + \eta)$ for some $v < q$ (note that $s_1 = (h + (\mathsf{sk}_0 \cdot \mathsf{sk}_1) \cdot r_x)/k_1 \pmod q$ in both cases). Conditioned on $\eta, \eta' < 2^s \cdot q - q$ (which is true except with probability 2^{-s}), the values $\eta, (v + \eta')$ are statistically indistinguishable—they are identically distributed in the range $[q, q \cdot 2^s - q]$, and either value falls outside of this range only with probability $q/(q \cdot 2^s) = 2^{-s}$. The lemma follows.

Corrupt P_1. The simulator $\mathcal{S}_{P1^*}$ proceeds as follows:

- **Setup.**
  1. On receiving $(\mathtt{prove}, \mathsf{sid}, \mathsf{sk}_1, \mathsf{pk}_1)$ from P_1 on behalf of $\mathcal{F}_{\mathsf{ComZK}}$, send $(\mathtt{share}, \mathsf{sid}, \mathsf{sk}_1)$ to $\mathcal{F}_{\mathsf{pECDSA}}^{\mathsf{CA}}$.
  2. On receiving $(\mathtt{pk\text{-}share}, \mathsf{sid}, \mathsf{pk}_0)$ from $\mathcal{F}_{\mathsf{pECDSA}}^{\mathsf{CA}}$, send $(\mathtt{r\text{-}committed}, \mathsf{sid}, \mathsf{pk}_0)$ to P_1 on behalf of $\mathcal{F}_{\mathsf{rVOLE}}$
  3. Wait to receive $(\mathtt{release}, \mathsf{sid})$ from P_1 on behalf of $\mathcal{F}_{\mathsf{ComZK}}$, and optionally $(\mathtt{predicate}, \mathsf{sid}, \phi)$ from P_1 on behalf of $\mathcal{F}_{\mathsf{rVOLE}}$, in which case forward $(\mathtt{predicate}, \mathsf{sid}, \phi')$ to $\mathcal{F}_{\mathsf{pECDSA}}^{\mathsf{CA}}$, where $\phi'(\mathsf{sk}) = \phi(\mathsf{sk}/\mathsf{sk}_1)$.
- **Partial Signing.** On receiving (sigid, h, r_x) from P_1, the values $(\alpha, \beta') \in \mathbb{Z}_M^2$ on behalf of $\mathcal{F}_{\mathsf{rVOLE}}$, and $(k_1, R_1 = k_1 \cdot G)$ on behalf of $\mathcal{F}_{\mathsf{ZK}}$, send $(\mathtt{sign\text{-}with}, \mathsf{sid}, \mathsf{sigid}, (k_1, h, r_x))$ to $\mathcal{F}_{\mathsf{pECDSA}}^{\mathsf{CA}}$. Additionally, define the predicate $\phi_{\mathsf{sigid}}(\mathsf{sk})$ as follows:

$$\phi_{\mathsf{sigid}}(\mathsf{sk}) = \begin{cases} 1 & \text{if } ((\mathsf{sk}/\mathsf{sk}_1) \cdot \alpha + \beta' \pmod M) \cdot R_1 = h \cdot G + r_x \cdot \mathsf{pk} \\ 0 & \text{otherwise} \end{cases}$$

and send $(\mathtt{predicate}, \mathsf{sid}, \mathsf{sigid}, \phi_{\mathsf{sigid}})$ to $\mathcal{F}_{\mathsf{pECDSA}}^{\mathsf{CA}}$.

Lemma 7.5. *The transcripts generated by $\mathcal{S}_{P1^*}$ are identical to that of a real execution.*

Once more, the setup phase is distributed identically in the real and simulated protocols as the values $(\mathsf{pk}_0, \mathsf{pk}_1, \mathsf{pk})$ are Diffie-Hellman triples, and there is a minor syntactic change in that ϕ is evaluated (on sk) within $\mathcal{F}_{\mathsf{pECDSA}}^{\mathsf{CA}}$ in the simulation rather than by $\mathcal{F}_{\mathsf{rVOLE}}$ (on input $\mathsf{sk}_0 = \mathsf{sk}/\mathsf{sk}_1$). The partial signing phase is equivalent up to syntax in the two executions; as $s_1' = \alpha \cdot \mathsf{sk}_0 + \beta' \pmod M$ by virtue of $\mathcal{F}_{\mathsf{rVOLE}}$ in the real protocol, and $s_1 = s_1' \pmod q$, the abort clause for P_0 is to check if $s_1 \cdot R_1 = (s_1' \pmod M) \cdot R_1$ matches $h \cdot G + r_x \cdot \mathsf{pk}$, which is exactly the predicate ϕ_{sigid}. The lemma, and hence the theorem, are therefore proven.

□

8 Unbiased Two-party ECDSA Signing

We now show how to use the partial ECDSA signing mechanism to construct realize (a predicated version of) the standard ECDSA signing functionality.

Functionality 8.1. $\mathcal{F}_{\mathsf{ECDSA}}^{\mathsf{CA}}(\mathcal{G})$: Predicated Two-party ECDSA

This functionality interacts with two parties, P_0, P_1. It also interacts directly with the ideal adversary $\mathcal{S}$, who can instruct the functionality to abort at any time. It is parameterized by an elliptic curve $\mathcal{G} = (\mathbb{G}, G, q)$ of prime order q and generated by G.
The superscript 'CA' indicates that it follows the 'conditional abort' paradigm. In particular, a corrupt P_1 may submit a predicate ϕ with each command, and induce an abort conditional on the secret key, i.e. if $\phi(\mathsf{sk}) = 0$.

KeyGen: On receiving $(\mathtt{keygen}, \mathsf{sid})$ from P_0, P_1 such that sid is previously unused,

1. Sample $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{ECDSAGen}(\mathbb{G}, G, q)$, and send $(\mathtt{public\text{-}key}, \mathsf{sid}, \mathsf{pk})$ to P_1.
2. If P_1 is corrupt, wait for $(\mathtt{predicate}, \mathsf{sid}, \phi)$ from $\mathcal{S}$, and if $\phi(\mathsf{sk}) = 0$ then send $(\mathtt{abort}, \mathsf{sid})$ to P_0 and halt.
3. Otherwise, store $(\mathsf{sid}, \mathsf{sk})$ in memory and send $(\mathtt{public\text{-}key}, \mathsf{sid}, \mathsf{pk})$ to P_0.

Signing: On receiving $(\mathtt{sign}, \mathsf{sid}, \mathsf{sigid}, m)$ from P_1, if $(\mathsf{sid}, \mathsf{sk})$ exists in memory, and $(\mathtt{abort}, \mathsf{sid})$ is not stored:

1. Send $(\texttt{sign}, \mathsf{sid}, \mathsf{sigid}, m)$ to P_0 and wait to receive $(\texttt{proceed}, \mathsf{sid}, \mathsf{sigid})$ in response.
2. Sample $k \leftarrow \mathbb{Z}_q$ and set $R = k \cdot G$
3. If P_1 is corrupt, send $(\texttt{nonce}, \mathsf{sid}, \mathsf{sigid}, R)$ to $\mathcal{S}$, and receive $(\texttt{predicate}, \mathsf{sid}, \mathsf{sigid}, \phi_{\mathsf{sigid}})$ in response. If $\phi_{\mathsf{sigid}}(\mathsf{sk}) = 0$, then store $(\texttt{abort}, \mathsf{sid})$ and send $(\texttt{abort}, \mathsf{sid})$ to P_0.
4. If no abort has occured, compute $s = \mathsf{ECDSASign}(\mathsf{sk}, m; k)$ and send $(\texttt{signature}, \mathsf{sid}, \mathsf{sigid}, m, (R, s))$ to P_0.

We give the protocol below.

Protocol 8.2. $\pi_{\mathsf{ECDSA}}(\mathcal{G})$**: Predicated Two-party ECDSA**

This protocol is executed between two parties, P_0, P_1. It is parameterized by an elliptic curve $\mathcal{G} = (\mathbb{G}, G, q)$ of prime order q and generated by G.

The protocol makes use of ideal oracles $\mathcal{F}_{\mathsf{pECDSA}}^{\mathsf{CA}}(\mathcal{G})$, $\mathcal{F}_{\mathsf{ComZK}}$, and $\mathcal{F}_{\mathsf{ZK}}$. Let H denote the hash function used by ECDSA.

KeyGen: On P_0 receiving $(\texttt{keygen}, \mathsf{sid})$ such that sid is previously unused for any input, it sends $(\texttt{setup}, \mathsf{sid})$ to $\mathcal{F}_{\mathsf{pECDSA}}^{\mathsf{CA}}$. Both parties wait to receive $(\texttt{pk-out}, \mathsf{sid}, \mathsf{pk})$ from $\mathcal{F}_{\mathsf{pECDSA}}^{\mathsf{CA}}$.

Signing: On P_1 receiving input $(\texttt{sign}, \mathsf{sid}, \mathsf{sigid}, m)$,

Message 1: P_1 samples $k_1 \leftarrow \mathbb{Z}_q$ and sends $(\texttt{prove}, \mathsf{sid}||\mathsf{sigid}||m, k_1, R_1 = k_1 \cdot G)$ to $\mathcal{F}_{\mathsf{ComZK}}$

Message 2: P_0 does the following:

1. Wait to receive $(\texttt{committed}, \mathsf{sid}||\mathsf{sigid}||m)$ from $\mathcal{F}_{\mathsf{ComZK}}$
2. Sample $k_0 \leftarrow \mathbb{Z}_q$, and sets $R_0 = k_0 \cdot G$
3. Send $(\texttt{prove}, \mathsf{sid}||\mathsf{sigid}, k_0, R_0)$ to $\mathcal{F}_{\mathsf{ZK}}$

Message 3: P_1 does the following:

4. Wait to receive $(\texttt{proven}, \mathsf{sid}||\mathsf{sigid}, R_0)$ from P_0
5. Set $R = k_1 \cdot R_0$ and parses r_x as its x-coordinate
6. Send $(\texttt{sign-with}, \mathsf{sid}, \mathsf{sigid}, (k_1, H(m), r_x))$ to $\mathcal{F}_{\mathsf{pECDSA}}^{\mathsf{CA}}$

Finalize: P_0 does the following:

7. Wait to receive $(\texttt{partial-sig}, \mathsf{sid}, \mathsf{sigid}, h, r'_x, R_1, s_1)$ from $\mathcal{F}_{\mathsf{pECDSA}}^{\mathsf{CA}}$
8. Compute $R = k_0 \cdot R_1$ and parse r_x as its x-coordinate.

9. Compute the putative signature $\sigma = (r_x, s_1/(k_0 + \delta))$
10. Verify that $h = H(m)$ and $r_x = r'_x$, and output the signature σ

Theorem 8.3. *Protocol $\pi_{\mathsf{ECDSA}}(\mathcal{G})$ realizes functionality $\mathcal{F}^{\mathsf{CA}}_{\mathsf{ECDSA}}(\mathcal{G})$ in the $\mathcal{F}_{\mathsf{ZK}}, \mathcal{F}_{\mathsf{ComZK}}, \mathcal{F}^{\mathsf{CA}}_{\mathsf{pECDSA}}$-hybrid model, in the presence of a malicious adversary statically corrupting up to one party.*

Proof. We give two simulators $\mathcal{S}_{P0^*}$ and $\mathcal{S}_{P1^*}$, to simulate the view of the adversary based on the corruption of parties P_0 and P_1 respectively.

Corrupt P_0. Simulator $\mathcal{S}_{P0^*}$ proceeds as follows:

- **Setup**. Upon receiving $(\texttt{public-key}, \mathsf{sid}, \mathsf{pk})$ from $\mathcal{F}^{\mathsf{CA}}_{\mathsf{ECDSA}}$, send $(\texttt{pk-out}, \mathsf{sid}, \mathsf{pk})$ to P_0 on behalf of $\mathcal{F}^{\mathsf{CA}}_{\mathsf{pECDSA}}$.
- **Signing**. Upon receiving $(\texttt{signature}, \mathsf{sid}, \mathsf{sigid}, m, (R, s))$ from $\mathcal{F}^{\mathsf{CA}}_{\mathsf{ECDSA}}$,
  1. Send $(\texttt{committed}, \mathsf{sid}||\mathsf{sigid}||m)$ to P_0 on behalf of $\mathcal{F}_{\mathsf{ComZK}}$
  2. Wait to receive $(\texttt{prove}, \mathsf{sid}||\mathsf{sigid}, k_0, R_0)$ from P_0 on behalf of $\mathcal{F}_{\mathsf{ZK}}$
  3. Compute the values $R_1 = k_0^{-1} \cdot R$ and $s_1 = s \cdot k_0$, and send $(\texttt{partial-sig}, \mathsf{sid}, \mathsf{sigid}, h, r'_x, R_1, s_1)$ to P_0 on behalf of $\mathcal{F}^{\mathsf{CA}}_{\mathsf{pECDSA}}$

Lemma 8.4. *The transcripts generated by simulator $\mathcal{S}_{P0^*}$ are distributed identically to the real protocol execution.*

This above lemma follows directly from the observations that each (R_0, R_1, R) are distributed identically in both executions as Diffie-Hellman tuples (where R_0 are first chosen by the adversary), and that s_1, s, k_0 are uniquely specified in both executions and are related as $s_1 = s \cdot k_0$.

Corrupt P_1. Simulator $\mathcal{S}$) proceeds as follows:

- **Setup**. Upon receiving $(\texttt{public-key}, \mathsf{sid}, \mathsf{pk})$ from $\mathcal{F}^{\mathsf{CA}}_{\mathsf{ECDSA}}$, send $(\texttt{pk-out}, \mathsf{sid}, \mathsf{pk})$ to P_1 on behalf of $\mathcal{F}^{\mathsf{CA}}_{\mathsf{pECDSA}}$.
- **Signing**. Upon receiving $(\texttt{prove}, \mathsf{sid}||\mathsf{sigid}||m, k_1, R_1 = k_1 \cdot G)$ from P_1,
  1. Send $(\texttt{sign}, \mathsf{sid}, \mathsf{sigid}, m)$ to $\mathcal{F}^{\mathsf{CA}}_{\mathsf{ECDSA}}$, and wait for $(\texttt{nonce}, \mathsf{sid}, \mathsf{sigid}, R)$ in response.
  2. Send $(\texttt{proven}, \mathsf{sid}||\mathsf{sigid}, R_0 = k_1^{-1} \cdot R_1)$ to P_1
  3. On receiving $(\texttt{sign-with}, \mathsf{sid}, \mathsf{sigid}, (k_1, h, r_x))$ such that $h = H(m)$, r_x is the x-coordinate of R, $k_1 \cdot G = R_1$, and optionally $(\texttt{predicate}, \mathsf{sid}, \mathsf{sigid}, \phi_{\mathsf{sigid}})$ from P_1 on behalf of $\mathcal{F}^{\mathsf{CA}}_{\mathsf{pECDSA}}$, forward $(\texttt{predicate}, \mathsf{sid}, \mathsf{sigid}, \phi_{\mathsf{sigid}})$ to $\mathcal{F}^{\mathsf{CA}}_{\mathsf{ECDSA}}$.

Lemma 8.5. *The transcripts generated by simulator $\mathcal{S}_{P1^*}$ are distributed identically to the real protocol execution.*

This above lemma follows directly from the observations that each (R_0, R_1, R) are distributed identically in both executions as random Diffie-Hellman tuples (where R_0 values are first chosen by the adversary). The theorem is hence proven.

□

9 Biased Two-party ECDSA Signing

We now give a more optimized protocol, with the tradeoff that it realizes a nonstandard ECDSA functionality. Essentially, the functionality allows for the adversary to bias the nonce, but includes a random offset value for each proposed adversarial bias. This corresponds to Doubly Enhanced Unforgeability as defined by Adjedj et al. [ABC+24], which as they showed holds in the generic group model. We recall the definition in Appendix B.

We give the functionality below along with the protocol, and discuss its implications for unforgeability in the next section.

Functionality 9.1. $\mathcal{F}_{\mathsf{ECDSA}}^{\mathsf{B},\mathsf{CA}}(\mathcal{G})$**: Biased Two-party ECDSA**

This functionality interacts with two parties, P_0, P_1. It also interacts directly with the ideal adversary $\mathcal{S}$, who can instruct the functionality to abort at any time. It is parameterized by an elliptic curve $\mathcal{G} = (\mathbb{G}, G, q)$ of prime order q and generated by G.
The superscript 'B, CA' indicates that it permits nonce bias, and operates in the 'conditional abort' paradigm. In particular, a corrupt P_1 may bias the signing nonce, and submit a predicate ϕ with each command to induce an abort conditionally on the secret key, i.e. abort if $\phi(\mathsf{sk}) = 0$.

KeyGen: On receiving $(\texttt{keygen}, \mathsf{sid})$ from P_0, P_1 such that sid is previously unused for any input,

1. Sample $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{ECDSAGen}(\mathbb{G}, G, q)$, store $(\mathsf{sid}, \mathsf{sk})$ in memory, and send $(\texttt{public-key}, \mathsf{sid}, \mathsf{pk})$ to P_1.
2. If P_1 is corrupt,
   (a) Wait to receive $(\texttt{predicate}, \mathsf{sid}, \phi)$ from $\mathcal{S}$.
   (b) If $\phi(\mathsf{sk}) = 0$, then store $(\texttt{abort}, \mathsf{sid})$ and send $(\texttt{abort}, \mathsf{sid})$ to P_0.
3. If no abort has occured, send $(\texttt{public-key},]\mathsf{sid}, \mathsf{pk})$ to P_0.

Signing: On receiving $(\texttt{sign}, \mathsf{sid}, \mathsf{sigid})$ from P_0, if $(\mathsf{sid}, \mathsf{sk})$ exists in memory, and $(\texttt{abort}, \mathsf{sid})$ is not stored:

1. If P_1 is not corrupt, upon receiving $(\texttt{sign}, \mathsf{sid}, \mathsf{sigid}, m)$ sample $k \leftarrow \mathbb{Z}_q$ and set $R = k \cdot G$

2. Otherwise if P_1 is corrupt,

   (a) Sample $k_0 \leftarrow \mathbb{Z}_q$ and set $R_0 = k_0 \cdot G$

   (b) Send $(\texttt{nonce}, \mathsf{sid}, \mathsf{sigid}, R_0)$ to $\mathcal{S}$

   (c) On receiving a fresh $(\texttt{nonce-bias}, \mathsf{sid}, \mathsf{sigid}, m, (\alpha_m \in \mathbb{Z}_q))$ from $\mathcal{S}$, sample and store $\beta_{m,\alpha_m} \leftarrow \mathbb{Z}_q$ and send $(\texttt{modifier}, \mathsf{sid}, \mathsf{sigid}, \alpha_m, \beta_j)$ to $\mathcal{S}$.

   (d) On receiving $(\texttt{bias-and-sign}, \mathsf{sid}, \mathsf{sigid}, (m, \alpha_m), \phi_{\mathsf{sigid}})$ from $\mathcal{S}$, set $k = \alpha_j \cdot (k_0 + \beta_j)$ and $R = k \cdot G$, where β_{m,α_m} is sampled afresh if it does not exist in memory.

   (e) If $\phi_{\mathsf{sigid}}(\mathsf{sk}) = 0$, then store $(\texttt{abort}, \mathsf{sid})$ and send $(\texttt{abort}, \mathsf{sid})$ to P_0.

3. If no abort has occured, compute $s = \mathsf{ECDSASign}(\mathsf{sk}, m; k)$ and send $(\texttt{signature}, \mathsf{sid}, \mathsf{sigid}, m, (R, s))$ to P_0.

Protocol 9.2. $\pi_{\mathsf{ECDSA}}(\mathcal{G}, p)$**: Biased Two-party ECDSA**

This protocol is executed between two parties, P_0, P_1. It is parameterized by an elliptic curve $\mathcal{G} = (\mathbb{G}, G, q)$ of prime order q and generated by G.

The protocol makes use of an ideal oracle $\mathcal{F}_{\mathsf{pECDSA}}^{\mathsf{CA}}(\mathcal{G})$, and random oracle $\mathsf{RO}_q : \{0,1\}^* \mapsto \mathbb{Z}_q$. Let H denote the hash function used by ECDSA.

KeyGen: On P_0 receiving $(\texttt{keygen}, \mathsf{sid})$ such that sid is previously unused for any input, it sends $(\texttt{setup}, \mathsf{sid})$ to $\mathcal{F}_{\mathsf{pECDSA}}^{\mathsf{CA}}$. Both parties wait to receive $(\texttt{pk-out}, \mathsf{sid}, \mathsf{pk})$ from $\mathcal{F}_{\mathsf{pECDSA}}^{\mathsf{CA}}$.

Signing: On P_0 receiving input $(\texttt{sign}, \mathsf{sid}, \mathsf{sigid}, m)$,
Message 1: P_0 does the following:

1. Sample $k_0 \leftarrow \mathbb{Z}_q$, and sets $R_0 = k_0 \cdot G$
2. Send $(\texttt{prove}, \mathsf{sid}||\mathsf{sigid}, R_0, k_0)$ to $\mathcal{F}_{\mathsf{ZK}}$

Message 2: P_1 does the following:

3. Wait to receive $(\texttt{proven}, \mathsf{sid}||\mathsf{sigid}, R_0)$ from P_0
4. Sample $k_1 \leftarrow \mathbb{Z}_q$, set $R_1 = k_1 \cdot G$, and compute $R' = k_1 \cdot R_0$
5. Set $R = k_1 \cdot (R_0 + \mathsf{RO}_q(\mathsf{sigid}, \mathsf{pk}, m, R_1, R') \cdot G)$ and parse r_x as its x-coordinate
6. Send $(\texttt{sign-with}, \mathsf{sid}, \mathsf{sigid}, (k_1, H(m), r_x))$ to $\mathcal{F}_{\mathsf{pECDSA}}^{\mathsf{CA}}$

Finalize: P_0 does the following:

7. Wait to receive $(\texttt{partial-sig}, \mathsf{sid}, \mathsf{sigid}, h, r'_x, R_1, s_1)$ from $\mathcal{F}_{\mathsf{pECDSA}}^{\mathsf{CA}}$
8. Define $\delta = \mathsf{RO}_q(\mathsf{sigid}, \mathsf{pk}, m, R_1, k_0 \cdot R_1)$, compute $R = (\delta + k_0) \cdot R_1$ and parse r_x as its x-coordinate.
9. Compute the putative signature $\sigma = (r_x, s_1/(k_0 + \delta))$
10. Verify that $h = H(m)$ and $r_x = r'_x$, and output the signature σ

Theorem 9.3. *Protocol $\pi_{\mathsf{ECDSA}}(\mathcal{G})$ realizes functionality $\mathcal{F}_{\mathsf{ECDSA}}^{\mathsf{B,CA}}(\mathcal{G})$ in the $\mathcal{F}_{\mathsf{ZK}}, \mathcal{F}_{\mathsf{pECDSA}}^{\mathsf{CA}}, \mathsf{RO}$-hybrid model and assuming the Knowledge of Exponent (KEA1) assumption, in the presence of a malicious adversary statically corrupting up to one party.*

Proof. We give two simulators $\mathcal{S}_{P0^*}$ and $\mathcal{S}_{P1^*}$, to simulate the view of the adversary based on the corruption of parties P_0 and P_1 respectively.

Corrupt P_0. Simulator $\mathcal{S}_{P0^*}$ proceeds as follows:

- **Setup**. Upon receiving $(\texttt{public-key}, \mathsf{sid}, \mathsf{pk})$ from $\mathcal{F}_{\mathsf{ECDSA}}^{\mathsf{CA}}$, send $(\texttt{pk-out}, \mathsf{sid}, \mathsf{pk})$ to P_0 on behalf of $\mathcal{F}_{\mathsf{pECDSA}}^{\mathsf{CA}}$.
- **Signing**. Upon receiving $(\texttt{signature}, \mathsf{sid}, \mathsf{sigid}, m, (R, s))$ from $\mathcal{F}_{\mathsf{ECDSA}}^{\mathsf{CA}}$,
  1. Wait to receive $(\texttt{prove}, \mathsf{sid}||\mathsf{sigid}, k_0, R_0)$ from P_0 on behalf of $\mathcal{F}_{\mathsf{ZK}}$
  2. Sample $\delta \leftarrow \mathbb{Z}_q$, compute $s_1 = s \cdot (k_0 + \delta)$ and $R_1 = (k_0 + \delta)^{-1} \cdot R$, and program $\mathsf{RO}_q(\mathsf{sigid}, \mathsf{pk}, m, R_1, k_0 \cdot R_1) = \delta$
  3. Send $(\texttt{partial-sig}, \mathsf{sid}, \mathsf{sigid}, h, r'_x, R_1, s_1)$ to P_0 on behalf of $\mathcal{F}_{\mathsf{pECDSA}}^{\mathsf{CA}}$

Lemma 9.4. *The transcripts generated by simulator $\mathcal{S}_{P0^*}$ are distributed indistinguishably from the real protocol execution.*

This above lemma follows directly from the observations that each $(R_0 + \delta \cdot G, R_1, R)$ are distributed identically in both executions as Diffie-Hellman tuples (where R_0 are first chosen by the adversary), and that s_1, s, k_0 are uniquely specified in both executions and are related as $s_1 = s \cdot k_0$, as well as the probability that the distinguisher queries $(\mathsf{sigid}, \mathsf{pk}, m, R_1, k_0 \cdot R_1)$ to RO_q prior to its programming being negligible.

Corrupt P_1. Simulator $\mathcal{S}_{P1^*}$ proceeds as follows:

- **Setup**. Upon receiving $(\texttt{public-key}, \mathsf{sid}, \mathsf{pk})$ from $\mathcal{F}_{\mathsf{ECDSA}}^{\mathsf{B,CA}}$, send $(\texttt{pk-out}, \mathsf{sid}, \mathsf{pk})$ to P_1 on behalf of $\mathcal{F}_{\mathsf{pECDSA}}^{\mathsf{CA}}$.
- **Signing**. Upon receiving $(\texttt{nonce}, \mathsf{sid}, \mathsf{sigid}, R_0)$ from $\mathcal{F}_{\mathsf{ECDSA}}^{\mathsf{B,CA}}$,
  1. Send $(\texttt{proven}, \mathsf{sid}||\mathsf{sigid}, R_0)$ to P_1 on behalf of $\mathcal{F}_{\mathsf{ZK}}$

2. Upon receiving a query of the form $(\mathsf{sigid}, \mathsf{pk}, m, R_1, k_1 \cdot R_0)$ on behalf of RO_q (where k_1 is obtained by invoking the Knowledge of Exponent extractor), send $(\texttt{nonce-bias}, \mathsf{sid}, \mathsf{sigid}, m, k_1)$ to $\mathcal{F}_{\mathsf{ECDSA}}^{\mathsf{B,CA}}$ and receive $(\texttt{modifier}, \mathsf{sid}, \mathsf{sigid}, k_1, \delta_{k_1})$ in response. Program $\mathsf{RO}_q(\mathsf{sigid}, \mathsf{pk}, m, R_1, k_1 \cdot R_0) = \delta_{k_1}$
3. On receiving $(\texttt{sign-with}, \mathsf{sid}, \mathsf{sigid}, (k_1', h, r_x))$ on behalf of $\mathcal{F}_{\mathsf{pECDSA}}^{\mathsf{CA}}$, if $h = H(m)$ and r_x is the x-coordinate of $R = k_1' \cdot (R_0 + \delta \cdot G)$, and optionally $(\texttt{predicate}, \mathsf{sid}, \mathsf{sigid}, \phi_{\mathsf{sigid}})$ as well, send $(\texttt{bias-and-sign}, \mathsf{sid}, \mathsf{sigid}, (m, k_1'), \phi_{\mathsf{sigid}})$ to $\mathcal{F}_{\mathsf{ECDSA}}^{\mathsf{B,CA}}$.

Lemma 9.5. *The transcripts generated by simulator $\mathcal{S}_{P1^*}$ are computationally indistinguishable from the real protocol execution.*

First note that $(\mathsf{sigid}, \mathsf{pk}, m, R_1, k_1 \cdot R_0)$ must be queried to RO_q with overwhelming probability, or P_0 will not obtain a valid signature. Conditioned on the success of the knowledge of exponent extractor (which is true with overwhelming probability), each set of $(R_0 + \delta \cdot G, R_1, R)$ are distributed identically in both executions as random Diffie-Hellman tuples (where R_1 values are first chosen by the adversary). The probability that the distinguisher queries $(\mathsf{sigid}, \mathsf{pk}, m, R_1, k_1 \cdot R_0)$ to RO_q prior to its programming is negligible, meaning that the transcript relative to RO_q is indistinguishable in both executions as well. Additionally, the fact that the ϕ predicates are evaluated within $\mathcal{F}_{\mathsf{ECDSA}}^{\mathsf{B,CA}}$ is merely a syntactic change. The lemma follows, and the theorem is hence proven.

□

10 Unforgeability

We now show how our predicated ECDSA signing functionalities retain the unforgeability guarantees of ECDSA, albeit with some security loss. We give a full proof for the more difficult case—Doubly Enhanced Unforgeability with biased nonces, as standard unforgeability with standard nonces follows easily from the same techniques.

We first describe our forgery experiment, which has a provision for a degree of bounded concurrency, parameterized by an integer c.

Experiment 10.1. $\textsc{Expt}_{\mathcal{A},\mathcal{G},\mathsf{c}}^{\mathcal{F}_{\mathsf{ECDSA}}^{\mathsf{B,CA}}}(1^\lambda)$

This experiment is run with an adversary $\mathcal{A}$ with unlimited sequential, and bounded concurrent invocations of $\mathcal{F}_{\mathsf{ECDSA}}^{\mathsf{B,CA}}(\mathcal{G})$. The functionality $\mathcal{F}_{\mathsf{ECDSA}}^{\mathsf{B,CA}}$ is instantiated with party P_1 marked corrupt. The concurrency parameter c determines the number of requests that $\mathcal{A}$ can make that return output independently of each other, i.e. overriding the global abort provision. We drop sid from queries to $\mathcal{F}_{\mathsf{ECDSA}}^{\mathsf{B,CA}}$ as there is only one instance.

Modelling bounded concurrency. The abort condition of $\mathcal{F}_{\mathsf{ECDSA}}^{\mathsf{B},\mathsf{CA}}$ is overridden and not honoured by default; instead a counter count indicates when a 'check for abort' instruction is due. Additionally, the signing oracle effectively terminates incomplete signing instances after c new instances have been initiated.

1. **Key Generation**.
   (a) Initialize $\mathsf{count} = 0$ and $\mathsf{max} = \infty$.
   (b) Send $(\texttt{keygen})$ to $\mathcal{F}_{\mathsf{ECDSA}}^{\mathsf{B},\mathsf{CA}}$ on behalf of both parties, and upon receiving $(\texttt{public-key}, \mathsf{pk})$, forward pk to $\mathcal{A}$.
   (c) Wait for the response $(\texttt{predicate}, \phi)$ from $\mathcal{A}$, and forward it to $\mathcal{F}_{\mathsf{ECDSA}}^{\mathsf{B},\mathsf{CA}}$.
   (d) If $\mathcal{F}_{\mathsf{ECDSA}}^{\mathsf{B},\mathsf{CA}}$ aborts, then do not permit any queries to the signing oracle.
2. **Signing Oracle**. Each signing instance offers three components, specified below:
   (a) **Initiate**. Upon receiving a signing query, increment count, send $(\texttt{sign}, \mathsf{count})$ to $\mathcal{F}_{\mathsf{ECDSA}}^{\mathsf{B},\mathsf{CA}}$ on behalf of P_0, and forward its response $(\texttt{nonce}, \mathsf{count}, R')$ to $\mathcal{A}$.
   (b) **Bias**. Upon receiving a message m and bias pair $(\alpha_j \in \mathbb{Z}_q)$ for a fresh j within the i^{th} signing request, send $(\texttt{nonce-bias}, i, j, \alpha_j)$ to $\mathcal{F}_{\mathsf{ECDSA}}^{\mathsf{B},\mathsf{CA}}$ on behalf of P_0, and forward its response $(\texttt{modifier}, i, j, \beta_j)$ to $\mathcal{A}$.
   (c) **Complete**. Upon receiving a finalized nonce bias (indexed by j) and a predicate ϕ for the i^{th} signing instance such that $\mathsf{count} - \mathsf{c} < i \leq \mathsf{max}$, send $(\texttt{bias-and-sign}, i, j, \phi_i)$ to $\mathcal{F}_{\mathsf{ECDSA}}^{\mathsf{B},\mathsf{CA}}$ on behalf of $\mathcal{S}$ and forward its output to $\mathcal{A}$. In the event that the response is an abort and $\mathsf{max} = \infty$, update $\mathsf{max} = \mathsf{count}$.
3. **Outcome**. Upon $\mathcal{A}$ supplying a putative forgery (m^*, σ^*) for a message m^* that was never queried to the signing oracle, output the result $\mathsf{ECDSAVerify}(\mathsf{pk}, m^*, \sigma^*)$.

We now prove that the Doubly Enhanced Unforgeability of ECDSA implies the same for our functionality as per our forgery experiment.

Theorem 10.2. *Let $\mathcal{A}$ be a PPT algorithm that succeeds in the forgery experiment $\textsc{Expt}_{\mathcal{A},\mathcal{G},\mathsf{c}}$ in time T with probability ε by making Q signing queries. Then, there is an adversary for Doubly Enhanced Unforgeability of ECDSA that succeeds in time T with probability $2^{-c}\varepsilon/Q$.*

Proof. We drop the superscript $\mathcal{F}_{\mathsf{ECDSA}}^{\mathsf{B},\mathsf{CA}}$ for readability. Observe that the challenger of the Doubly Enhanced Unforgeability experiment essentially

implements the same interface as $\text{EXPT}_{\mathcal{A},\mathcal{G},\mathsf{c}}$, with the exception of the $\{\phi_i(\mathsf{sk})\}_{i\in[Q]}$ predicate evaluations that $\mathcal{F}^{\mathsf{B,CA}}_{\mathsf{ECDSA}}$ offers. Constructing a reduction to Doubly Enhanced Unforgeability given an adversary $\mathcal{A}$ for $\text{EXPT}_{\mathcal{A},\mathcal{G},\mathsf{c}}$ is therefore a matter of simulating the $\{\phi_i(\mathsf{sk})\}_{i\in[Q]}$ evaluations. As the reduction can not actually have the secret key in order to evaluate each $\phi_i(\mathsf{sk})$, we must employ an alternative strategy to answer such queries. In particular, we will show—through two intermediate experiments—that the predicate evaluations can be simulated with noticeable probability by a simple guessing strategy.

Experiment $\text{EXPT}^{*}_{\mathcal{A},\mathcal{G},\mathsf{c}}$: This experiment is identical to $\text{EXPT}_{\mathcal{A},\mathcal{G},\mathsf{c}}$, with the exception that the value max^* (corresponding to max in EXPT) is sampled uniformly from the range $[1, Q]$. Observe that $\Pr[\mathsf{max}^* = \mathsf{max}] \geq 1/Q$, which directly implies that:

$$\Pr[\text{EXPT}^{*}_{\mathcal{A},\mathcal{G},\mathsf{c}} = 1] \geq \frac{1}{Q} \cdot \Pr[\text{EXPT}_{\mathcal{A},\mathcal{G},\mathsf{c}} = 1]$$

Experiment $\text{EXPT}^{**}_{\mathcal{A},\mathcal{G},\mathsf{c}}$: This experiment is identical to $\text{EXPT}^{*}_{\mathcal{A},\mathcal{G},\mathsf{c}}$, with the exception that it samples a vector $\{\mathbf{r}_i \leftarrow \{0,1\}\}_{i\in[\mathsf{c}]}$ and the evaluation of the predicate $\phi_i(\mathsf{sk})$ is replaced with the following predicate:

$$\phi^{**}_{\mathbf{r}}(i) = \begin{cases} \mathbf{r}_{\mathsf{max}-i} & \text{for } i \in (\mathsf{max} - c, \mathsf{max}] \\ 1 & \text{otherwise} \end{cases}$$

with ϕ during key generation replaced by $\phi^{**}_{\mathbf{r}}(0)$. Observe that by definition the first aborting query is indexed greater than $\mathsf{max} - \mathsf{c}$, and so it holds that $\phi_i(\mathsf{sk}) = \phi^{**}_{\mathbf{r}}(i) = 1$ for all $i \leq \mathsf{max} - c$. The remaining queries to reason about are those in the range $i \in (\mathsf{max} - \mathsf{c}, \mathsf{max}]$, and as $\mathbf{r}$ is sampled uniformly it holds that $\Pr[\phi_i(\mathsf{sk}) = \phi^{**}_{\mathbf{r}}(i)\ \forall i \in [\mathsf{max}]] = 2^{-\mathsf{c}}$. Therefore as $\text{EXPT}^{**}_{\mathcal{A},\mathcal{G},\mathsf{c}}$ and $\text{EXPT}^{*}_{\mathcal{A},\mathcal{G},\mathsf{c}}$ are distributed identically when $\phi^{**}_{\mathbf{r}}(i) = \phi_i(\mathsf{sk})$ for each $i \in [\mathsf{max}]$, it holds that:

$$\Pr[\text{EXPT}^{**}_{\mathcal{A},\mathcal{G},\mathsf{c}} = 1] \geq 2^{-\mathsf{c}} \cdot \Pr[\text{EXPT}^{*}_{\mathcal{A},\mathcal{G},\mathsf{c}} = 1]$$

We now provide the final experiment—the reduction to Doubly Enhanced Unforgeability—below explicitly for completeness.

Experiment 10.3. $\text{EXPT}^{\mathsf{Ch\text{-}DEU}}_{\mathcal{A},\mathcal{G},\mathsf{c}}(1^\lambda)$

This experiment interacts with the challenger for the Doubly Enhanced Unforgeability experiment for ECDSA, denoted Ch-DEU. See Appendix B for the definition of this challenger. Changes relative to the original experiment $\text{EXPT}_{\mathcal{A},\mathcal{G},\mathsf{c}}(1^\lambda)$ that are not merely syntactic, are underlined.

1. **Key Generation**.
   (a) Initialize $\mathsf{count} = 0$, and sample $\{\mathbf{r}_i\}_{i\in[\mathsf{c}]}$ and $\mathsf{max} \leftarrow [Q]$.

Define predicate $\underline{\phi^{**}_{\mathbf{r}}(i)} = \begin{cases} \mathbf{r}_{\mathsf{max}-i} & \text{for } i \in (\mathsf{max}-c, \mathsf{max}] \\ 1 & \text{otherwise} \end{cases}$

(b) Query Ch-DEU for the public key pk, and upon receiving it forward pk to $\mathcal{A}$.

(c) Wait for the response $(\texttt{predicate}, \phi)$ from $\mathcal{A}$.

(d) If $\underline{\phi^{**}_{\mathbf{r}}(0) = 0}$, then do not permit any queries to the signing oracle.

2. **Signing Oracle**. Each signing instance offers three components, specified below:

   (a) **Initiate**. Upon receiving a signing query, increment count, request Ch-DEU for a pre-signature, and forward its response $(\texttt{nonce}, \mathsf{count}, R')$ to $\mathcal{A}$.

   (b) **Bias**. Upon receiving $(m, R_0, \alpha_j \in \mathbb{Z}_q)$ for a previously sent R', forward it to Ch-DEU, and forward its response $(\texttt{modifier}, i, j, \beta_j)$ to $\mathcal{A}$.

   (c) **Complete**. Upon receiving a finalized nonce bias (indexed by j) and a predicate ϕ for the i^{th} signing instance such that $\mathsf{count} - \mathsf{c} < i \leq \mathsf{max}$, if $\underline{\phi^{**}_{\mathbf{r}}(i) = 0 \text{ then abort, otherwise}}$ request Ch-DEU for a signature with bias α_j and forward its output to $\mathcal{A}$.

3. **Outcome**. Upon $\mathcal{A}$ supplying a putative forgery (m^*, σ^*) for a message m^* that was never queried to the signing oracle, output the result $\mathsf{ECDSAVerify}(\mathsf{pk}, m^*, \sigma^*)$.

It is clear to see that $\text{Expt}^{\mathsf{Ch\text{-}DEU}}_{\mathcal{A},\mathcal{G},\mathsf{c}}$ is in fact the Doubly Enhanced Unforgeability experiment for ECDSA with additional restrictions, and that a successful forgery in this experiment corresponds to a successful adversary for Doubly Enhanced Unforgeability with the same running time and success probability. As $\text{Expt}^{\mathsf{Ch\text{-}DEU}}_{\mathcal{A},\mathcal{G},\mathsf{c}}$ differs from $\text{Expt}^{**}_{\mathcal{A},\mathcal{G},\mathsf{c}}$ only syntactically, we have that:

$$\Pr[\text{Expt}^{\mathsf{Ch\text{-}DEU}}_{\mathcal{A},\mathcal{G},\mathsf{c}} = 1] = \Pr[\text{Expt}^{**}_{\mathcal{A},\mathcal{G},\mathsf{c}} = 1] \geq \frac{2^{-\mathsf{c}}}{Q} \cdot \Pr[\text{Expt}_{\mathcal{A},\mathcal{G},\mathsf{c}} = 1]$$

and as none of the intermediate experiments inflate the running time, this proves the theorem.

□

Corollary 10.4. *When ECDSA is instantiated with a generic group $\mathcal{G}$, there is no efficient forger that is given arbitrary access to $\mathcal{F}^{\mathsf{B,CA}}_{\mathsf{ECDSA}}(\mathcal{G})$, even with the additional privilege of being allowed (at any time) to adaptively formulate up to $\mathsf{c} \in \log(\lambda)$ concurrent queries that override the global abort provision.*

The above corollary follows from Doubly Enhanced Unforgeability of ECDSA [ABC+24, Theorem 2.1], and the reduction to Doubly Enhanced Unforgeability of ECDSA in Theorem 10.2.

11 Efficiency

We first describe a simple optimization that one can make when considering the protocol as whole when instantiated (rather than through the abstractions in the earlier sections).

Extra Degree of Freedom in PCF. Our $\mathbb{Z}_M$ VOLE protocol is essentially generates a random $\mathbb{Z}_M$ VOLE correlation $(a, b), (x, a{\cdot}x{+}b)$ that is derandomized to chosen inputs $(\alpha, \beta), (x, \alpha \cdot x + \beta)$ by sending the differences $\delta_a = \alpha - a$ and $\delta_b = \beta - b$. However, our ECDSA signing protocol has an unused degree of freedom in the choice of these inputs: recall that the inputs are $\alpha = \mathsf{sk}_1/k_1$ and $\beta' = \beta + q \cdot \eta$, where $\beta = H(m)/k \pmod q$. Observe that β' is statistically indistinguishable from a $\mathbb{Z}_{q \cdot 2^{\mathsf{s}}}$ value whose residue modulo q is β, and there is no particular constraint on k_1—implying that k_1 can be fixed as a function of β rather than the other way around. In particular, rather than derandomize b from the $\mathbb{Z}_M$ VOLE correlation, P_0 can instead set $k_1 = (b/H(m)) \pmod q$. This does not harm correctness by causing an overflow modulo M, as $b < M - q^2$ except with probability $2^{-\mathsf{s}}$, or security as $\lfloor b/q \rfloor$ is at least as large an integer one-time pad than η in the protocol.

This optimization cuts the bandwidth cost of the $\mathbb{Z}_M$ VOLE in our signing protocol by half.

Setup Protocol Cost. We only give a brief overview of the cost of key generation, as it is a one-time operation. The dominant cost is that of executing r OTs to set up each $\mathcal{F}_{\mathsf{rsVOLE}}(\mathbf{p}_i)$ instance; each $\mathcal{F}_{\mathsf{rsVOLE}}(\mathbf{p}_i)$ requires a $\binom{\mathbf{p}_i}{\mathbf{p}_i - 1}$-OT instance, which in turn can be instantiated with $\log \mathbf{p}_i$ $\binom{2}{1}$-OTs. The range proof component requires another $|q|$ $\binom{2}{1}$-OTs. This brings the total to about $|q| + \sum_i \log \mathbf{p}_i = 3|q| + \mathsf{s}$ $\binom{2}{1}$-OT instances, the cost of which can be amortized with OT extension if required. Additionally, the range proof mechanism transmits a ciphertext $\mathbf{ct}$ of size $\lambda \cdot |q| \cdot r$ bits.

Signing Protocol. We enumerate all costs for a 256-bit signing curve, and a statistical security parameter $\mathsf{s} = 80$. Signing requires the following components:

- $\mathcal{F}_{\mathsf{ZK}}$: We can use the standard Sigma protocol for Schnorr's proof of knowledge of discrete logarithm, compiled into a non-interactive proof with the Fiat-Shamir transformation. In case the Knowledge of Exponent assumption can be made (as our biased signing protocol does), a proof consists of a single group element that completes a Diffie-Hellman tuple. The Fiat-Shamir proof costs one group element, and one ($|q|/2$ sized) scalar.

- $\mathcal{F}_{\mathsf{rVOLE}}$: As discussed earlier, a $\mathcal{F}_{\mathsf{rVOLE}}$ instance in the context of our signing protocol requires transmitting a single $\mathbb{Z}_M$ value. As $M \approx q^2 \cdot 2^{\mathsf{s}}$, this costs $2|q| + \mathsf{s}$ bits.

Our unbiased signing protocol therefore has the following characteristics:

- Three messages of interaction.
- P_0 to P_1: one $\mathbb{G}$ element and a NIZK proving knowledge of its discrete logarithm, for a total of 96 bytes.
- P_1 to P_0: one commitment, one $\mathbb{G}$ element (with proof of knowledge), and one $\mathbb{Z}_M$ element, for a total of 186 bytes.

Our optimized, biased signing protocol has the following characteristics:

- Two messages of interaction.
- P_0 to P_1: one $\mathbb{G}$ element and a KEA1-based NIZK proving knowledge of its discrete logarithm, for a total of 64 bytes.
- P_1 to P_0: one $\mathbb{G}$ element, and one $\mathbb{Z}_M$ element, for a total of 106 bytes.

Of course, a full implementation will include a robust mechanism to convey and agree on session information, which may require more data. As is standard, we only report the cost of the signing protocol in isolation.

Empirical Efficiency. A Rust implementation of our scheme requires 2.2ms to sign a message on a laptop with an Apple M1 processor.

12 Extensions

We now briefly discuss extensions of our results.

12.1 Preprocessing and Pipelining

Several works in the literature consider a "pre-signing" mode of operation [CGG+20, DOK+20, DJN+20, GS22, DKLs24] wherein the OLE and the nonce sampling phase can be executed before the message is known, so that signing can be a non-interactive information-theoretic operation once the message is available. Our protocols can be adapted to support this operation as well, by preprocessing the $\mathbb{Z}_M$ to $\mathbb{Z}_q$ VOLE translation. However, as with previous works, this induces an extra assumption on ECDSA itself, which can be justified in the generic group model [GS22]. Doerner et al. [DKLs24] proposed a "pipelined" mode of operation wherein each signing session generates the first message of the next session in parallel, which saves a round on average with no extra complexity assumption. Our three round protocol supports this mode of operation as well.

As our protocol is built on a PCF, it also supports non-interactive preprocessing of the bulk of the computation required for each signing instance. To our knowledge, ours is the first ECDSA signing protocol to support this feature without an apriori bound on the number of instances.

12.2 Extension to the $(2, n)$ Setting

It is straightforward to include backup nodes to store shares of the signing key, by having the two parties in our protocol execute a standard resharing of the signing key. It is then also possible to configure the system so that any pair of the n parties can execute distributed signing, by incorporating the range proof mechanism in our setup protocol. However in the latter case, care must be taken to have all parties immediately cease all signing sessions with any party that triggers an abort, as maintaining the global abort clause is crucial for security.

12.3 Tight Security With an Ad-hoc Assumption

Our security proof, like that of Lindell's [Lin17] incurs a multiplicative loss proportional to the number of signatures. However, there is not necessarily a matching attack that degrades security by this factor. In fact, Lindell showed under an ad-hoc assumption that their protocol permits a tight simulation, and hence no security loss relative to the unforgeability of ECDSA. Recall that the security loss (in both works) comes from having to simulate the evaluation of the predicate $\phi_{\mathsf{sigid}}(\mathsf{sk})$ to the adversary—the proof essentially guesses which sigid will induce the first $\phi_{\mathsf{sigid}}(\mathsf{sk}) = 0$, and hence abort the protocol. Lindell's assumption (which pertains to the predicate permitted by their specific protocol) is essentially that for any efficient adversary, $\phi_{\mathsf{sigid}}(\mathsf{sk})$ can be replaced undetected with $\phi_{\mathsf{sigid}}(\mathsf{sk}')$ for a completely independent sk' value sampled by the simulator.

Their protocol permits a predicate that checks if $a \cdot \mathsf{sk}_0 + b \pmod N = a \cdot \mathsf{sk}_0 + b \pmod q$ for an adversarially chosen $a, b \in \mathbb{Z}_N$ where N is an RSA modulus and sk_0 is P_0's share, given that $\mathsf{sk}_0 \cdot G$ is public. Consequently, their assumption (called "Paillier-EC") is that replacing $\mathsf{sk}_0 \cdot G$ with an independent $\mathsf{sk}_0' \leftarrow \mathbb{Z}_q$ while keeping the predicate evaluation unchanged, is undetectable to an efficient adversary. We can formulate a similar assumption called $\mathsf{CRT\text{-}EC}$ for our setting, that no efficient adversary can induce the following experiment to output 1 with noticeable probability:

Experiment $\mathsf{CRT\text{-}EC}_{\mathcal{A},\mathcal{G}}(1^\lambda)$:

1. Sample $x_0, x_1 \leftarrow \mathbb{Z}_q$ and $e \leftarrow \{0, 1\}$, and set $X = x_0 \cdot G$
2. Define oracle $\mathcal{O}_e(a, b, c, d)$:
   - if $a \cdot x_e + b \pmod M \pmod q = c \cdot x_e + d \pmod q$ then return 1.
   - otherwise halt and do not respond to further queries.

3. Obtain $c' \leftarrow \mathcal{A}^{\mathcal{O}_e(\cdot)}(X)$ and output 1 iff $c = c'$

It is straightforward to remove the security loss that depends on the number of signing queries in our reduction to unforgeability (Theorem 10.2) under the CRT-EC assumption: rather than guessing which evaluation of ϕ_i will be the first to return 0, the reduction can instead simulate the evaluation of ϕ_i itself as:

$$\phi_i(\mathsf{sk}') = \begin{cases} 1 & \text{if } ((\mathsf{sk}'/\mathsf{sk}_1) \cdot \alpha + \beta \bmod M) \bmod q = (r_x/k_1) \cdot \mathsf{sk}' + h/k_1 \bmod q \\ 0 & \text{otherwise} \end{cases}$$

where $\mathsf{sk}' \leftarrow \mathbb{Z}_q$ is sampled by the reduction, and the rest of the values are as per the equivalent predicate ϕ_{sigid} defined by Simulator $\mathcal{S}_{P1^*}$ in the proof of Theorem 7.3. It is easy to see that evaluating the above predicate on sk' instead of sk is undetectable to an efficient adversary under the CRT-EC assumption. This yields the following theorem:

Theorem 12.1. *Let $\mathcal{A}$ be a PPT algorithm that succeeds in the forgery experiment $\text{EXPT}_{\mathcal{A},\mathcal{G},\mathsf{c}}$ in time T with probability ε by making Q signing queries. Then, under the CRT-EC assumption, there is an adversary for Doubly Enhanced Unforgeability of ECDSA that succeeds in time T with probability $2^{-c}\varepsilon$.*

13 Acknowledgements

The author is deeply grateful to Vladyslav Khomenko for implementing and benchmarking the protocol, and for useful comments on the writeup. The author would also like to thank Naman Kumar for insightful discussions during the course of writing this paper.

References

[ABC+24] Michael Adjedj, Constantin Blokh, Geoffroy Couteau, Arik Galansky, Antoine Joux, and Nikolaos Makriyannis. Two-round 2PC ECDSA at the cost of 1 OLE. Cryptology ePrint Archive, Report 2024/1950, 2024.

[ALSZ17] Gilad Asharov, Yehuda Lindell, Thomas Schneider, and Michael Zohner. More efficient oblivious transfer extensions. *Journal of Cryptology*, 30(3):805–858, July 2017.

[ANO+22] Damiano Abram, Ariel Nof, Claudio Orlandi, Peter Scholl, and Omer Shlomovits. Low-bandwidth threshold ECDSA via pseudorandom correlation generators. In *2022 IEEE Symposium on Security and Privacy*, pages 2554–2572. IEEE Computer Society Press, May 2022.

[BCG+19] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators: Silent OT extension and more. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 489–518. Springer, Cham, August 2019.

[BCG+20] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Correlated pseudorandom functions from variable-density LPN. In *61st FOCS*, pages 1069–1080. IEEE Computer Society Press, November 2020.

[BHLS25] Dan Boneh, Iftach Haitner, Yehuda Lindell, and Gil Segev. Exponent-VRFs and their applications. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part VII*, volume 15607 of *LNCS*, pages 195–224. Springer, Cham, May 2025.

[BP04] Mihir Bellare and Adriana Palacio. The knowledge-of-exponent assumptions and 3-round zero-knowledge protocols. In Matthew Franklin, editor, *CRYPTO 2004*, volume 3152 of *LNCS*, pages 273–289. Springer, Berlin, Heidelberg, August 2004.

[Can00] Ran Canetti. Security and composition of multiparty cryptographic protocols. *Journal of Cryptology*, 13(1):143–202, January 2000.

[Can01] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd FOCS*, pages 136–145. IEEE Computer Society Press, October 2001.

[CCD+20] Megan Chen, Ran Cohen, Jack Doerner, Yashvanth Kondi, Eysa Lee, Schuyler Rosefield, and abhi shelat. Multiparty generation of an RSA modulus. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part III*, volume 12172 of *LNCS*, pages 64–93. Springer, Cham, August 2020.

[CCL+19] Guilhem Castagnos, Dario Catalano, Fabien Laguillaumie, Federico Savasta, and Ida Tucker. Two-party ECDSA from hash proof systems and efficient instantiations. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 191–221. Springer, Cham, August 2019.

[CCL+20] Guilhem Castagnos, Dario Catalano, Fabien Laguillaumie, Federico Savasta, and Ida Tucker. Bandwidth-efficient threshold EC-DSA. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020, Part II*, volume 12111 of *LNCS*, pages 266–296. Springer, Cham, May 2020.

[CGG+20] Ran Canetti, Rosario Gennaro, Steven Goldfeder, Nikolaos Makriyannis, and Udi Peled. UC non-interactive, proactive, threshold ECDSA with identifiable aborts. In Jay Ligatti, Xinming Ou,

Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020*, pages 1769–1787. ACM Press, November 2020.

[CHHK25] Geoffroy Couteau, Carmit Hazay, Aditya Hegde, and Naman Kumar. ømega $(1/\lambda)$-rate boolean garbling scheme from generic groups. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2025, Proceedings, Part IV*, volume 16003 of *Lecture Notes in Computer Science*, pages 489–521. Springer, 2025.

[DHIM25] Jack Doerner, Iftach Haitner, Yuval Ishai, and Nikolaos Makriyannis. From OT to OLE with subquadratic communication. Cryptology ePrint Archive, Paper 2025/1722, 2025.

[DJN+20] Ivan Damgård, Thomas Pelle Jakobsen, Jesper Buus Nielsen, Jakob Illeborg Pagter, and Michael Bæksvang Østergaard. Fast threshold ECDSA with honest majority. In Clemente Galdi and Vladimir Kolesnikov, editors, *SCN 20*, volume 12238 of *LNCS*, pages 382–400. Springer, Cham, September 2020.

[DKLs18] Jack Doerner, Yashvanth Kondi, Eysa Lee, and abhi shelat. Secure two-party threshold ECDSA from ECDSA assumptions. In *2018 IEEE Symposium on Security and Privacy*, pages 980–997. IEEE Computer Society Press, May 2018.

[DKLs19] Jack Doerner, Yashvanth Kondi, Eysa Lee, and abhi shelat. Threshold ECDSA from ECDSA assumptions: The multiparty case. In *2019 IEEE Symposium on Security and Privacy*, pages 1051–1066. IEEE Computer Society Press, May 2019.

[DKLs24] Jack Doerner, Yashvanth Kondi, Eysa Lee, and abhi shelat. Threshold ECDSA in three rounds. In *2024 IEEE Symposium on Security and Privacy*, pages 3053–3071. IEEE Computer Society Press, May 2024.

[DOK+20] Anders P. K. Dalskov, Claudio Orlandi, Marcel Keller, Kris Shrishak, and Haya Shulman. Securing DNSSEC keys via threshold ECDSA from generic MPC. In Liqun Chen, Ninghui Li, Kaitai Liang, and Steve A. Schneider, editors, *ESORICS 2020, Part II*, volume 12309 of *LNCS*, pages 654–673. Springer, Cham, September 2020.

[DPSZ12] Ivan Damgård, Valerio Pastro, Nigel P. Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 643–662. Springer, Berlin, Heidelberg, August 2012.

[GG18] Rosario Gennaro and Steven Goldfeder. Fast multiparty threshold ECDSA with fast trustless setup. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018*, pages 1179–1194. ACM Press, October 2018.

[Gil99] Niv Gilboa. Two party RSA key generation. In Michael J. Wiener, editor, *CRYPTO'99*, volume 1666 of *LNCS*, pages 116–129. Springer, Berlin, Heidelberg, August 1999.

[GJKR96] Rosario Gennaro, Stanislaw Jarecki, Hugo Krawczyk, and Tal Rabin. Robust threshold DSS signatures. In Ueli M. Maurer, editor, *EUROCRYPT'96*, volume 1070 of *LNCS*, pages 354–371. Springer, Berlin, Heidelberg, May 1996.

[GS22] Jens Groth and Victor Shoup. On the security of ECDSA with additive key derivation and presignatures. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part I*, volume 13275 of *LNCS*, pages 365–396. Springer, Cham, May / June 2022.

[HLNR23] Iftach Haitner, Yehuda Lindell, Ariel Nof, and Samuel Ranellucci. Fast secure multiparty ECDSA with practical distributed key generation and applications to cryptocurrency custody. Cryptology ePrint Archive, Paper 2018/987, 2023.

[HMRT22] Iftach Haitner, Nikolaos Makriyannis, Samuel Ranellucci, and Eliad Tsfadia. Highly efficient OT-based multiplication protocols. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part I*, volume 13275 of *LNCS*, pages 180–209. Springer, Cham, May / June 2022.

[IKNP03] Yuval Ishai, Joe Kilian, Kobbi Nissim, and Erez Petrank. Extending oblivious transfers efficiently. In Dan Boneh, editor, *CRYPTO 2003*, volume 2729 of *LNCS*, pages 145–161. Springer, Berlin, Heidelberg, August 2003.

[KOR23] Yashvanth Kondi, Claudio Orlandi, and Lawrence Roy. Two-round stateless deterministic two-party Schnorr signatures from pseudorandom correlation functions. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part I*, volume 14081 of *LNCS*, pages 646–677. Springer, Cham, August 2023.

[KOS15] Marcel Keller, Emmanuela Orsini, and Peter Scholl. Actively secure OT extension with optimal overhead. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *CRYPTO 2015, Part I*, volume 9215 of *LNCS*, pages 724–741. Springer, Berlin, Heidelberg, August 2015.

[KOS16] Marcel Keller, Emmanuela Orsini, and Peter Scholl. MASCOT: Faster malicious arithmetic secure computation with oblivious transfer. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel,

Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016*, pages 830–842. ACM Press, October 2016.

[Lan95] Susan K. Langford. Threshold dss signatures without a trusted party. In Don Coppersmith, editor, *CRYPTO'95*, volume 963 of *LNCS*, pages 397–409. Springer, Berlin, Heidelberg, August 1995.

[Lin17] Yehuda Lindell. Fast secure two-party ECDSA signing. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part II*, volume 10402 of *LNCS*, pages 613–644. Springer, Cham, August 2017.

[LL23] Yehuda Lindell and Jeff Lunglhofer. The subtleties of error handling flaws in mpc. https://www.coinbase.com/en-sg/blog/the-subtleties-of-error-handling-flaws-in-mpc, August 2023. Coinbase Blog, accessed 2025-09-27.

[LN18] Yehuda Lindell and Ariel Nof. Fast secure multiparty ECDSA with practical distributed key generation and applications to cryptocurrency custody. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018*, pages 1837–1854. ACM Press, October 2018.

[MR01] Philip D. MacKenzie and Michael K. Reiter. Two-party generation of DSA signatures. In Joe Kilian, editor, *CRYPTO 2001*, volume 2139 of *LNCS*, pages 137–154. Springer, Berlin, Heidelberg, August 2001.

[MYG24] Nikolaos Makriyannis, Oren Yomtov, and Arik Galansky. Practical key-extraction attacks in leading MPC wallets. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024*, pages 3053–3064. ACM Press, October 2024.

[OSY21] Claudio Orlandi, Peter Scholl, and Sophia Yakoubov. The rise of paillier: Homomorphic secret sharing and public-key silent OT. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part I*, volume 12696 of *LNCS*, pages 678–708. Springer, Cham, October 2021.

[Roy22] Lawrence Roy. SoftSpokenOT: Quieter OT extension from small-field silent VOLE in the minicrypt model. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part I*, volume 13507 of *LNCS*, pages 657–687. Springer, Cham, August 2022.

[RS21] Lawrence Roy and Jaspal Singh. Large message homomorphic secret sharing from DCR and applications. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part III*, volume 12827 of *LNCS*, pages 687–717, Virtual Event, August 2021. Springer, Cham.

[ST19] Nigel P. Smart and Younes Talibi Alaoui. Distributing any elliptic curve based protocol. In Martin Albrecht, editor, *17th IMA International Conference on Cryptography and Coding*, volume 11929 of *LNCS*, pages 342–366. Springer, Cham, December 2019.

[TS21] Dmytro Tymokhanov and Omer Shlomovits. Alpha-rays: Key extraction attacks on threshold ECDSA implementations. Cryptology ePrint Archive, Report 2021/1621, 2021.

[vdP23] Joop van de Pol. Dont overextend your oblivious transfer. https://blog.trailofbits.com/2023/09/20/dont-overextend-your-oblivious-transfer/, September 2023. Trail of Bits Blog, accessed 2025-09-27.

A Helper Functionalities

We first give the OT functionality. Note that it combines two types of usage: select all-but-one message, as well as select one out of two messages when $p = 2$.

Functionality A.1. $\mathcal{F}_{\mathsf{OT}}(p)$**: Oblivious Transfer**

This functionality interacts with two parties, S, R. It is parameterized by an integer p. As this is a two-party functionality, all outputs are adversarially delayed.

Choose: On receiving $(\mathsf{sid}, \texttt{choose-all-but}, c \in \mathbb{Z}_p)$ from R such that sid is previously unused, send $(\mathsf{sid}, \texttt{chosen})$ to S and store (sid, c) in memory.

Choose: On receiving $(\mathsf{sid}, \texttt{choose}, c \in \{0, 1\})$ from R such that sid is previously unused and $p = 2$, send $(\mathsf{sid}, \texttt{chosen})$ to S and store $(\mathsf{sid}, 1 - c)$ in memory.

Send: On receiving $(\mathsf{sid}, \texttt{messages}, \{\boldsymbol{\mu}_i\}_{i \in \mathbb{Z}_p})$ from S, if (sid, c) is in memory, then send $(\mathsf{sid}, \texttt{outputs}, \{\boldsymbol{\mu}_i\}_{i \in \mathbb{Z}_p \setminus \{c\}})$ to R

Next, we give the committed zero-knowledge functionality.

Functionality A.2. $\mathcal{F}_{\mathsf{ComZK}}(\mathcal{G})$**: Committed Zero-knowledge**

This functionality interacts with two parties, P, V. It is parameterized by an elliptic curve $\mathcal{G} = (\mathbb{G}, G, q)$ generated by G and of prime order q.

Commit: On receiving $(\texttt{prove}, \mathsf{sid}, x, X)$ from P for a previously unused sid, store (sid, x, X) send $(\texttt{comitted}, \mathsf{sid})$ to V

Reveal: On receiving $(\texttt{release}, \mathsf{sid})$ from P for an (sid, x, X) in memory, if $x \in \mathbb{Z}_q$ and $X \in \mathbb{G}$ and $x \cdot G = X$, send $(\texttt{proven}, \mathsf{sid}, X)$ to V.

Finally, we give the zero-knowledge functionality.

Functionality A.3. $\mathcal{F}_{\mathsf{ZK}}^{\mathcal{G}}$**: Zero-knowledge**

This functionality interacts with two parties, P, V. It is parameterized by an elliptic curve $\mathcal{G} = (\mathbb{G}, G, q)$ generated by G and of prime order q.

Prove: On receiving $(\texttt{prove}, \mathsf{sid}, x \in \mathbb{Z}_q, X \in \mathbb{G})$ from P for a previously unused sid, if $x \cdot G = X$ then send $(\texttt{proven}, \mathsf{sid}, X)$ to V.

B Doubly Enhanced Unforgeability

We recall below the definition of Doubly Enhanced Unforgeability of ECDSA, as formulated by Adjedj et al. [ABC+24].

Experiment B.1. $\textsc{Expt}_{\mathcal{A},\mathcal{G}}(1^\lambda)$

This experiment is run with an adversary $\mathcal{A}$ with unlimited access to a signing oracle.

1. **Key Generation**. Sample $\mathsf{sk} \leftarrow \mathbb{Z}_q$ and send $\mathsf{pk} = \mathsf{sk} \cdot G$ to $\mathcal{A}$
2. **Signing Oracle**. Each signing instance offers three components, specified below:
   (a) **Initiate**. Upon receiving a signing query, sample $k_0 \leftarrow \mathbb{Z}_q$, set $R_0 = k_0 \cdot G$, and send $(\texttt{nonce}, \mathsf{count}, R_0)$ to $\mathcal{A}$.
   (b) **Bias**. Upon receiving tuple $(m, R_0, \alpha_j \in \mathbb{Z}_q)$ for a fresh j, sample $\beta_j \leftarrow \mathbb{Z}_q$ and send $(\texttt{modifier}, i, j, \beta_j)$ to $\mathcal{A}$.
   (c) **Complete**. Upon receiving a finalized $(R_0, \alpha_j, m, \beta_j)$, compute $s = (H(m) + r_x \cdot \mathsf{sk})/(\alpha_j k_0 + \beta_j))$
3. **Outcome**. Upon $\mathcal{A}$ supplying a putative forgery (m^*, σ^*) for a message m^* that was never queried to the signing oracle, output the result $\mathsf{ECDSAVerify}(\mathsf{pk}, m^*, \sigma^*)$.

Definition B.2.

The ECDSA signature scheme instantiated with elliptic curve $\mathcal{G}$ satisfies Doubly Enhanced Unforgeability if for every PPT adversary, $\Pr[\textsc{Expt}_{\mathcal{A},\mathcal{G}}(1^\lambda) = 1] \in \mathrm{negl}(\lambda)$.